1-2
DLD
CSE-1203
MSH

09.07.2023

for Lab: + Logisim simulator
CSE-1204

5 digit number → $d_4 d_3 d_2 d_1 d_0$

LSD:
d_0 is the LSD

MSD:
d_4 is the MSD

---

byte = 8 bits
(short) word = 2 bytes
(double) word = 4 bytes
(long) word = 8 bytes

---

d → binary conversion : → mental conversion

$(1564)_{10}$

$1564 - \frac{1024}{2^{10}} = 540$

$540 - \frac{512}{2^9} = 28$

$28 - (2^8 / 2^7 / 2^6 / 2^5 / 2^4) = (-ve)$

$28 - \frac{16}{2^4} = 12$

$12 - 8 = 4 \quad 4 - 4 = 0 \quad 0 - 2 = (V)$

1	1	0	0	0	0	1	1	1	0

CSE1203
DLD
MSH

16.07.23

radix conversions :

octal ⟶ hexadecimal can't be done directly

Half Adder : ⟶ where additions are done without the carry

Full Adder : ⟶ where addition are done with all the carry bits

radix complement :

- subtraction by addition
- multiplication by addition
- division by addition

* Advantage and disadvantage of 1's and 2's complements.
  - ± 0's appear

CSE 12083
MMAH

19.07.23

* AOI (And, Or, Inverter) logic gates can be replaced by NAND gates.

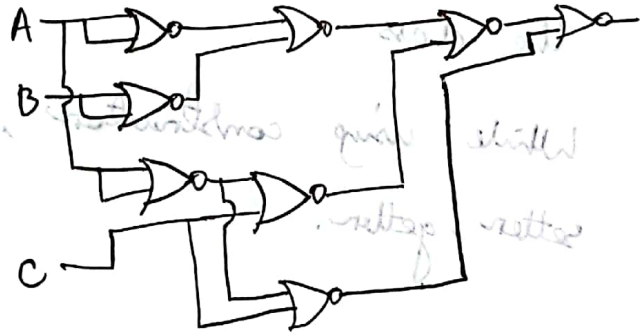
$$\overline{(AB+A\bar{C}).(\bar{A}+C)}$$

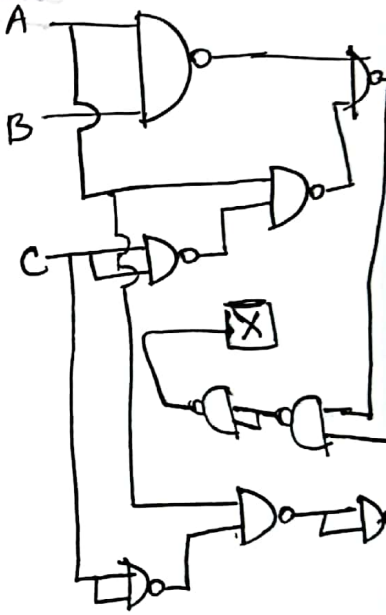
$$\overline{(AB+A\bar{C})}.\overline{\bar{A}+C}$$

$$\overline{(\overline{AB}.\overline{A\bar{C}}).(A.\bar{C})}$$

B A

$$\overline{\left(\overline{(\bar{A}+\bar{B})}+\overline{(\bar{A}+C)}\right)+\overline{(\bar{A}+C)}}$$





---

* NOR gates and NAND gates are functionally complete.

* Variable: A symbol used to represent a logical quantity

Complement: the inverse of a variable and is indicated

CSE 1203
DLD
MSH

23.07.23

* n সংখ্যক digit number কে BCD conversion করলে 4n সংখ্যক digit number পাওয়া যায়।

Gray Code

→ অন্য code

* Gray Code is also called
  - distance code → from a number to its next, the difference is only 1 bit
  - reflect code
  - cyclic code

⇒ Why we use/need 'Gray Code'?

↳ To remove delay in logic gate implementations of codes.

To generation n bit gray code, we will mirror (n-1) bits

generating 2 bit: first we take, 0 0 — 0

3 bit: 0 0 1 — 1

right 1 against 0 in previous mirror 0 1 1 — 2

0 1 0 — 3

1 1 0 — 4

1 1 1 — 5

1 0 1 — 6

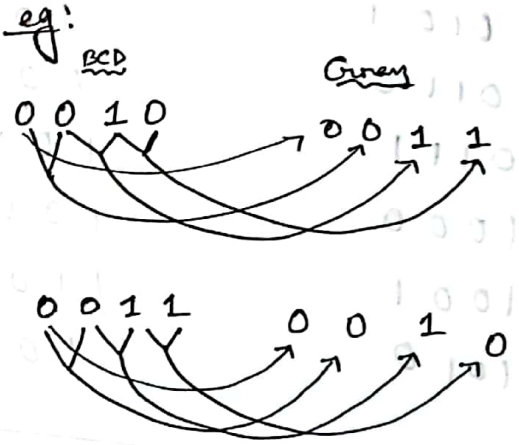
1 0 0 — 7

Conversion from BCD to Gray :

BCD	Gray
0000	0000
0001	0001
0010	0011
0011	0010
0100	0110
0101	0111
0110	0101
0111	0100
1000	1100
1001	1101

* Prefix (MSB - leftbit) will be copied directly
* Then we see in BCD, if we find any change in the 2 side-by-side bits of the BCD, we put 1 in Gray, otherwise 0.

eg:



Gray to BCD :

Gray	BCD
0000	0000
0001	0001
0011	0010
0010	0011
0110	
0111	

* MSB Prefix is fixed
* Then we see in Gray numbers from leftmost ^ we check if it is 0, then there is no change in bit from the previous bit, If 1, then there is change in bit from the previous.

Excess-3 code :

(BCD) + 11

→ self complimenting number

→ the bits are already in 1's compliment here,

complimenting 1 → 0011

we get → 1100 → 9

BCD	Excess-3
0000	0011
0001	0100
0010	0101
0011	0110
0100	0111
0101	1000
0110	1001
0111	1010
1000	1011
1001	1100
1010	1101

Weighted code :

Why use? → we want to secure the information sharing

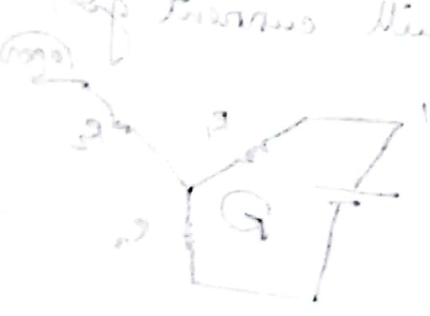
8, 4, -2, -1 weighted

8	4	-2	-1	
0	0	0	0	→ 0
0	0	1	1	→ 1
0	1	1	0	→ 2
0	1	0	1	→ 3
0	1	0	0	→ 4

	8	4	-2	-1
5 →	1	0	1	1
6 →	1	0	1	0
7 →	1	0	0	1
8 →	1	0	0	0
9 →	1	1	1	1
10 →	1	1	1	0

Biquinary: only 2 bits can be put

5	0	4	3	2	1	0	
0	1	0	0	0	0	1	0
0	1	0	0	0	1	0	1
0	1	0	0	1	0	0	2
0	1	0	1	0	0	0	3
0	1	1	0	0	0	0	4
1	0	1	0	0	0	0	5
1	0	0	0	0	1	0	6
1	0	0	0	1	0	0	7
1	0	0	1	0	0	0	8
1	0	1	0	0	0	0	9



CSE 1st
DLD
SH

30.07.23

Boolean Algebra:

↳ Closure: after doing any operation, the values we get will be in the domain (boolean algebra maintains it always)

↳ Associative law: $a \oplus (b \oplus c) = (a \oplus b) \oplus c$

$\oplus$ → any binary operator

we can do this because associative law permits this

↳ Commutative law: $A \oplus B = B \oplus A$

$\oplus$ → any binary operator

↳ Identity law: $A \cdot 1 = A$

$A + 0 = A$

↳ Inverse law: if $A \in$ set X, then $A^{-1} \in$ set X

$(A + \bar{A}) = 1$

↳ Distributive law: when I will work on 2 operators like, $A \cdot (B + C)$, the operators in between the operands can interchange.

$A \cdot (B+C) = (A \cdot B) + (A \cdot C)$

Boolean Algebra *SPECIAL* :

* Duality :

as → $A \cdot 1 = A$

$A + 0 = A$

↳ this is duality in boolean algebra

if from a boolean equation, we invert the operations ($\cdot \rightarrow +$, $+ \rightarrow \cdot$) and invert the constant values ($1 \rightarrow 0$, $0 \rightarrow 1$), the resulting equation is also TRUE

Theorem 1(a): $\boxed{X + X = X}$

↳ proof :

$(X + X) = (X + X) \cdot 1$ [Identity law]

$= (\overset{A}{X} + \overset{B}{X})(\overset{A}{X} + \overline{X})$ [Inverse law]

$= X + X \cdot \overline{X}$

$= X + 0 = X$

duality : $\boxed{X \cdot X = X}$

Theorem 6(a) : $\boxed{X + XY = X}$

$X + XY = X(1 + Y)$ [Distributive]

$= X \cdot 1$ [Identity (a)]

$= X$ [Identity (b)]

∴ duality : $\boxed{X \cdot (X + Y) = X}$

Simplify $XY + \bar{X}Z + YZ$.

$= XY + \bar{X}Z + YZ = XY + \bar{X}Z + YZ + YZ$ [

$= XY + Z(\bar{X} + Y) + XY + YZ + \bar{X}Z + YZ$

$= Y(X+Z) + Z(X$

$\Rightarrow XY + \bar{X}Z + YZ$

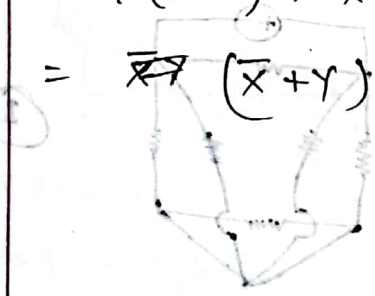
$= X\bar{X} + XY + \bar{X}Z + YZ$

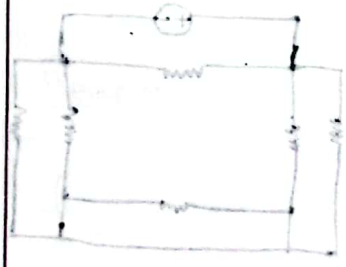
$= \bar{X}Y + YZ + \bar{X}Z + X\bar{X}$

$= Y(X+Z) + \bar{X}(X+Z)$

$= (\bar{X} + Y)(X+Z)$

DLD
Morris Mano
↳ 2nd 2.4 → 2.5
Reading



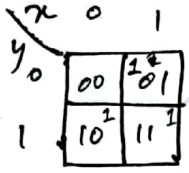


CSE 1203
DLD
5H

06.08.23

① Map method: using k-map is to create [Boolean expression]

↳ Karnaugh map → K-map

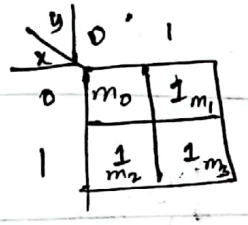


adjacent is the only value which is a value that has a change of only 1 value

x	y	
0	0	0
0	1	1
1	0	1
1	1	0

⇒ place the minterms on the map

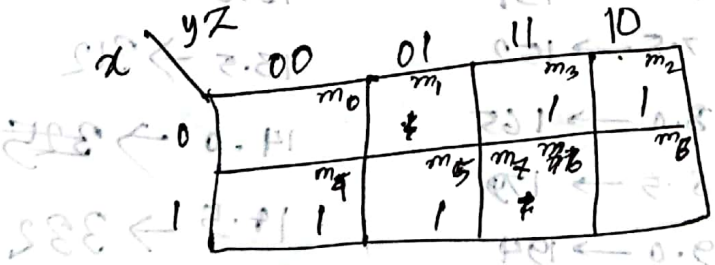
⇒ grouping → only with 1, 2, 4 values

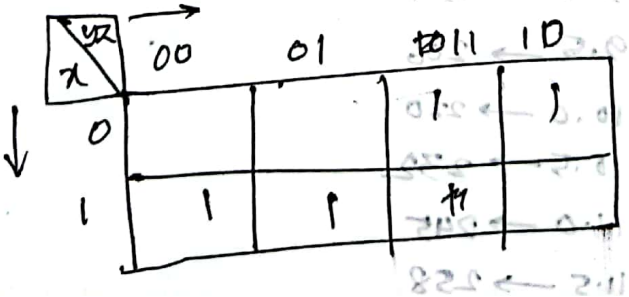


$$f = \bar{x}y + x\bar{y} + xy$$
$$= \bar{x}y + xy + x\bar{y} + xy$$
$$= y(x+\bar{x}) + x(y+\bar{y})$$
$$= x + y$$

~~$m_1 + m_2 + m_3$~~

$= m_1 + m_3 + m_2 + m_3$





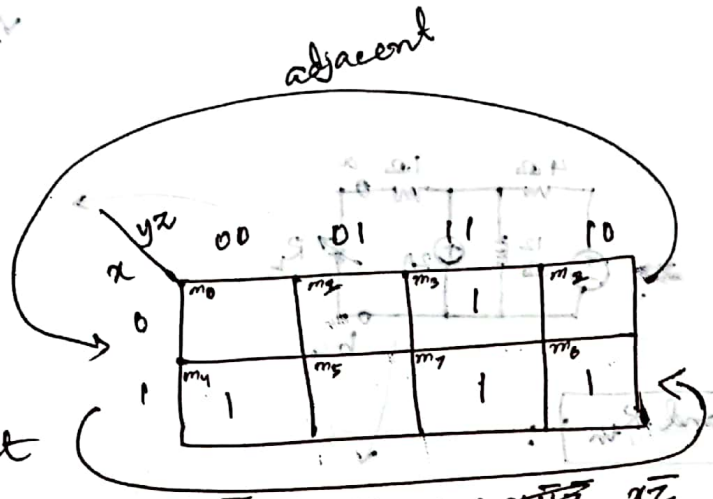
	f	x	y	z
m_0	0	0	0	0
m_1	0	0	0	1
m_2	1	0	1	0
m_3	1	0	1	1
m_4	1	1	0	0
m_5	1	1	0	1
m_6	0	1	1	0
m_7	0	1	1	1

* now take the minterms and groups them

↳ $x\bar{y}^2 + \bar{x}y$

$\Sigma(3,4,6,7)$

x	y	z	
0	0	0	
0	0	1	
0	1	0	
0	1	1	1
1	0	0	1
1	0	1	
1	1	0	1
1	1	1	1



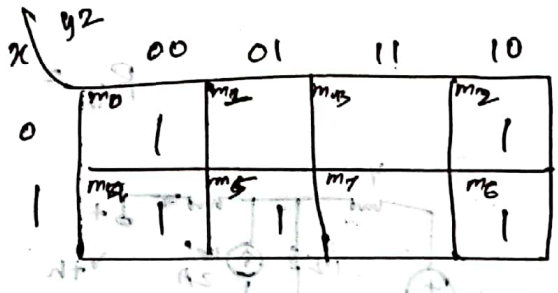
$= \bar{x}yz + xy + x\bar{y}\bar{z} + x\bar{z}$

$(m_3 + m_4) + (m_7 + m_6) + (m_4 + m_6)$

PRACTICE

K-MAP

3 variable



$\Sigma(0,2,4,5,6)$

$= (m_0 + m_4) + (m_4 + m_5) + (m_4 + m_6) + (m_2 + m_6) + (m_2 + m_0)$

$= \bar{y}\bar{z} + x\bar{y} + x\bar{z} + y\bar{z} + \bar{x}\bar{z}$

CSE 1203

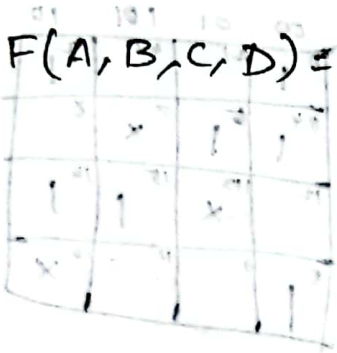
16, 08, 23

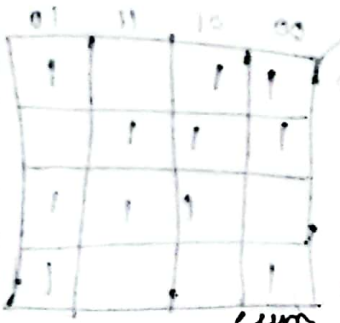
$F(A,B,C,D) = \prod(3,4,6,7,11,12,13,14,15)$

↳ This means there are 0 in these Max terms

$F(A,B,C,D) = \sum(1,2,5,8,9,10)$

↳ This means there are 1 in these

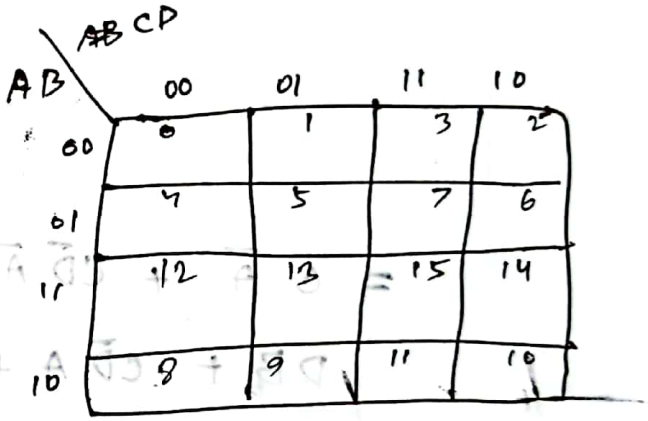


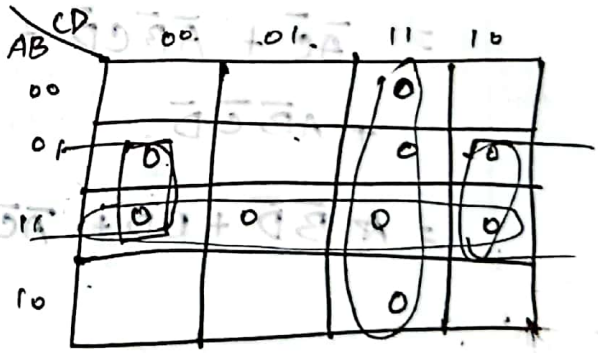


sum

Product of maxterms =

$(\bar{C}+\bar{D}$



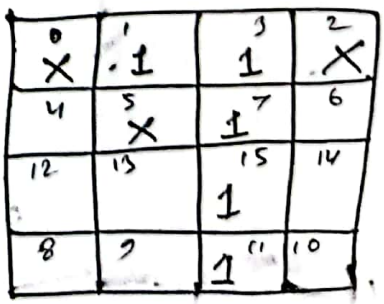


$F(w,x,y,z) = \sum(1,3,7,11,15)$

don't care X → 0, X → 1

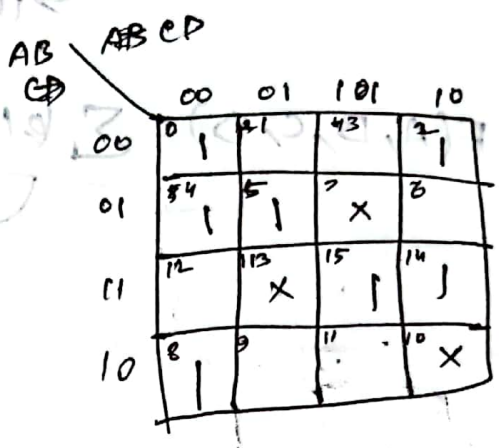
$d(w,x,y,z) = \sum(0,2,5)$

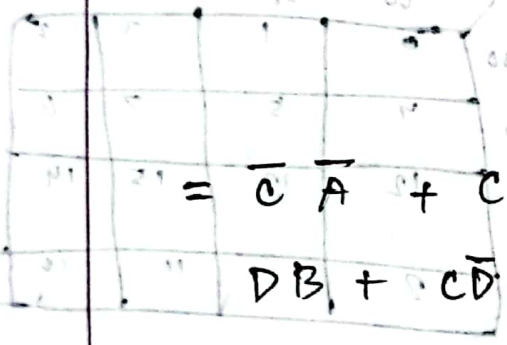
we can take X as 1 or 0



$F(A,B,C,D) = \Sigma(0,2,4,5,8,14,15)$

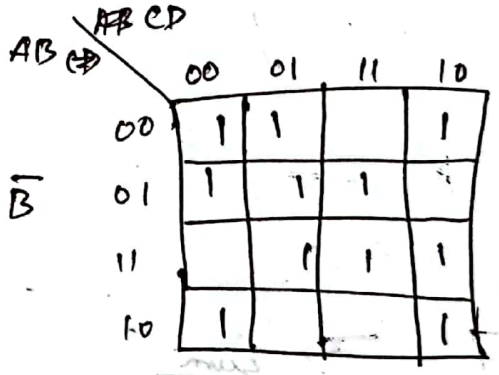
$d(A,B,C,D) = \Sigma(7,10,13)$





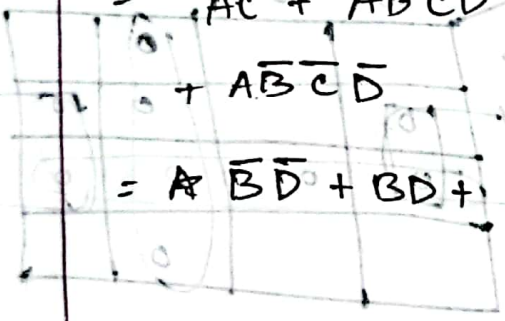
$= \bar{C}\bar{A} + C\bar{D}\bar{A}\bar{B} +$

$DB + C\bar{D}A + \cancel{A\bar{B}}\ \bar{C}\bar{D}A\bar{B}$



$= \bar{A}\bar{C} + \bar{A}\bar{B}C\bar{D} + BD + AC\bar{D}$

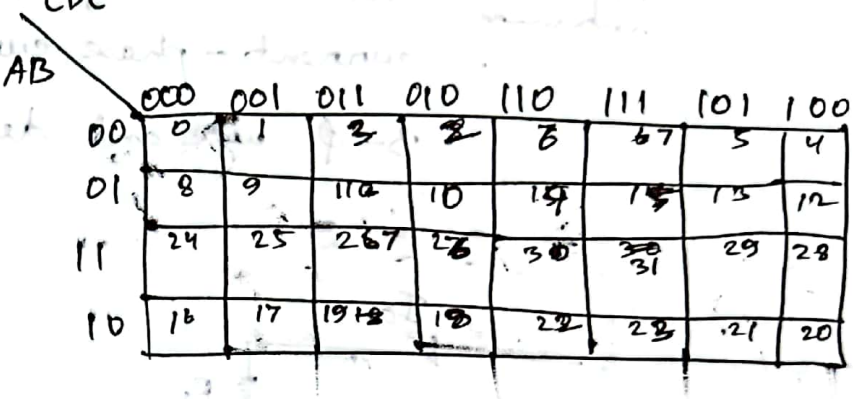
$+ A\bar{B}\bar{C}\bar{D}$



$= A\bar{B}\bar{D} + BD + \bar{A}\bar{C}\bar{D} + AC\bar{D}$

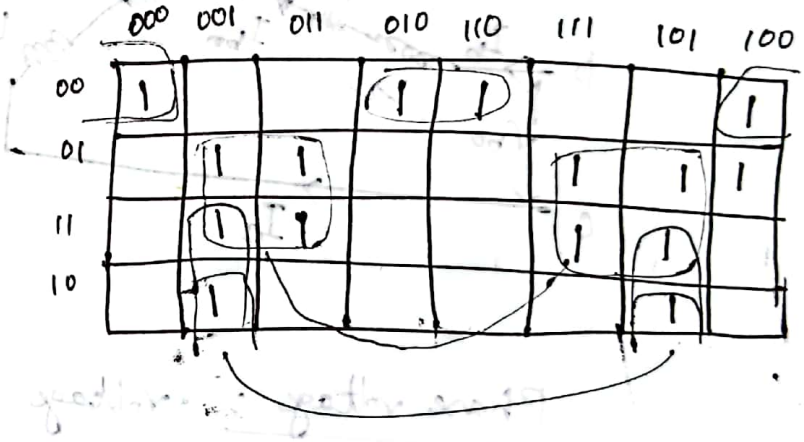
4 Variable OVER

5 Variable:



$F(A,B,C,D,E)$

$= \Sigma(0,2,4,6,9,11,12,13,15,17,21,25,27,29,31)$



CSE 1203
MBH

20.08.2023

5 variable to 6 variable is going to get us a big (beeeeeeg) K map. So we made

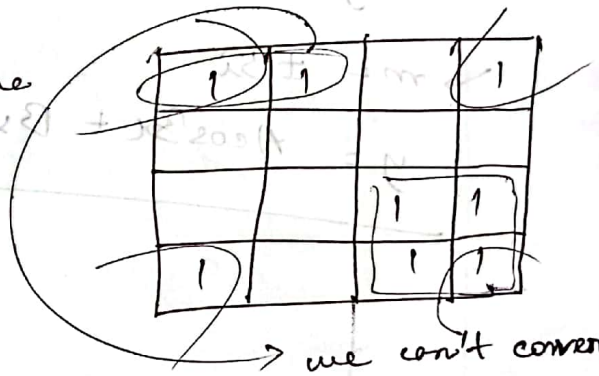
→ programmable method or → Tabulation method

from the minterms we are given, we count the number of 1's in our minterms.

→ works with "essential prime implicant"

→ group of 1's

finds out, after we group terms



→ we can't cover this '1' without ~~covering~~ this group

F(w, x, y, z) = Σ(1, 4, 6, 7, 8, 9, 10, 11, 15)

0001	1
0100	4
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1111	15

Table 1 :

group	minterms	
1	1	0001
	4	0100
	8	~~0111~~ 1000
2	6	
	9	
	10	
3	7	
	11	
4	15	

Table 2 : find differences of terms in Table 1:

Group	minterms	
1	(1-9)	(-001) 8
	(4-6)	(01-0) 2
	(8-9)	(100-) 1
	(8-10)	(10-0) 2
2	(6-7)	(011-) 1
	(9-11)	(10-1) 2
	(10-11)	(101-) 1
3	(7-15)	(-111) 8
	(11-15)	(1-11) 4

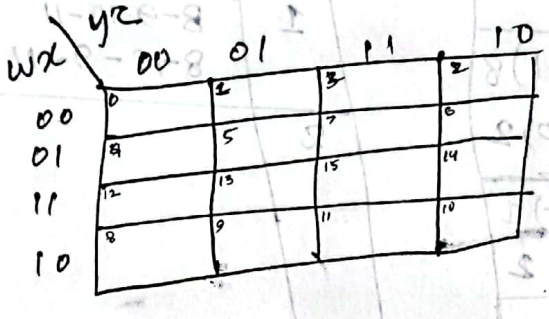
Table 3:

Group	minterms	representation
1	8-9-10-11	10--
	8-10-9-11	10--
2		

	1	4	6	7	8	9	10	11	15
wx'					×	×	×	×	
$x'y'z$	×					×			
~~x'y~~ $w'xz'$		×	×						
$w'xy$			×	×					
xyz				×					×
wyz								×	×

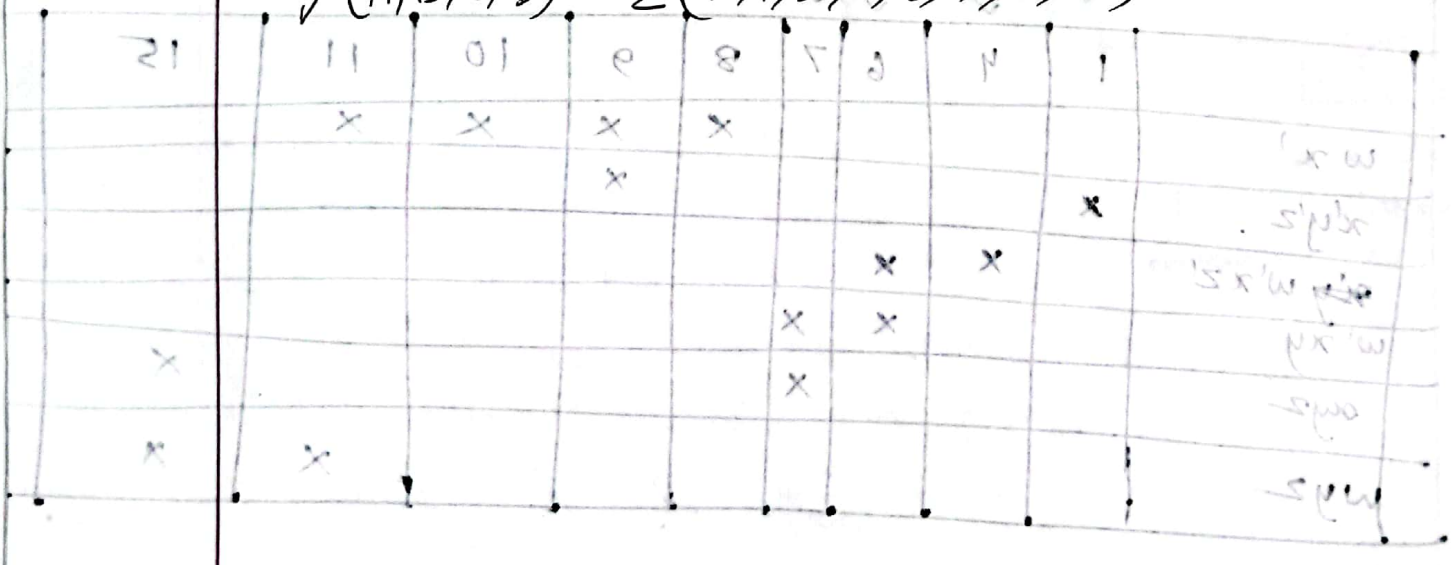
$$F = wx' + x'y'z + w'xz' + xyz$$

K map: (for f):



HOMEWORK

$$Y(A,B,C,D) = \Sigma(0,1,3,7,8,9,11,15)$$



Topic Name : (DLD) CSE 1203 (SH) Day : Sunday

Time : 9.40 Date : 10 / 09 / 23

BCD sum using 4 bit binary adder

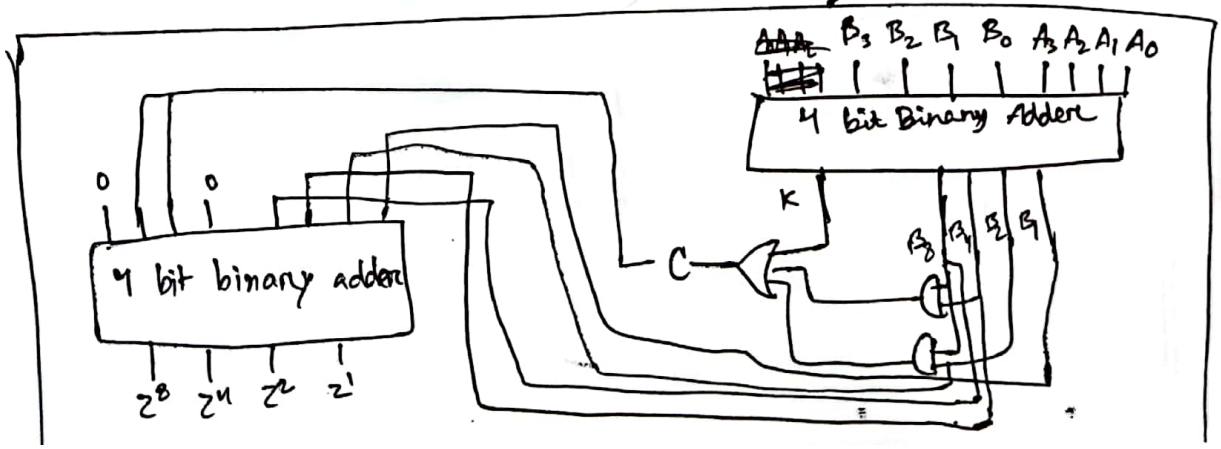
K	Binary sum	C	BCD sum
0	1 0 1 0	1	0 0 0 0
0	1 0 1 1	1	0 0 0 1
0	1 1 0 0	1	0 0 1 0
0	1 1 0 1	1	0 0 1 1
0	1 1 1 0	1	0 1 0 0
0	1 1 1 1	1	0 1 0 1
1	0 0 0 0	1	0 1 1 0
1	0 0 0 1	1	0 1 1 1
1	0 0 1 0	1	1 0 0 0
1	0 :	1	1 0 0 1

10

19

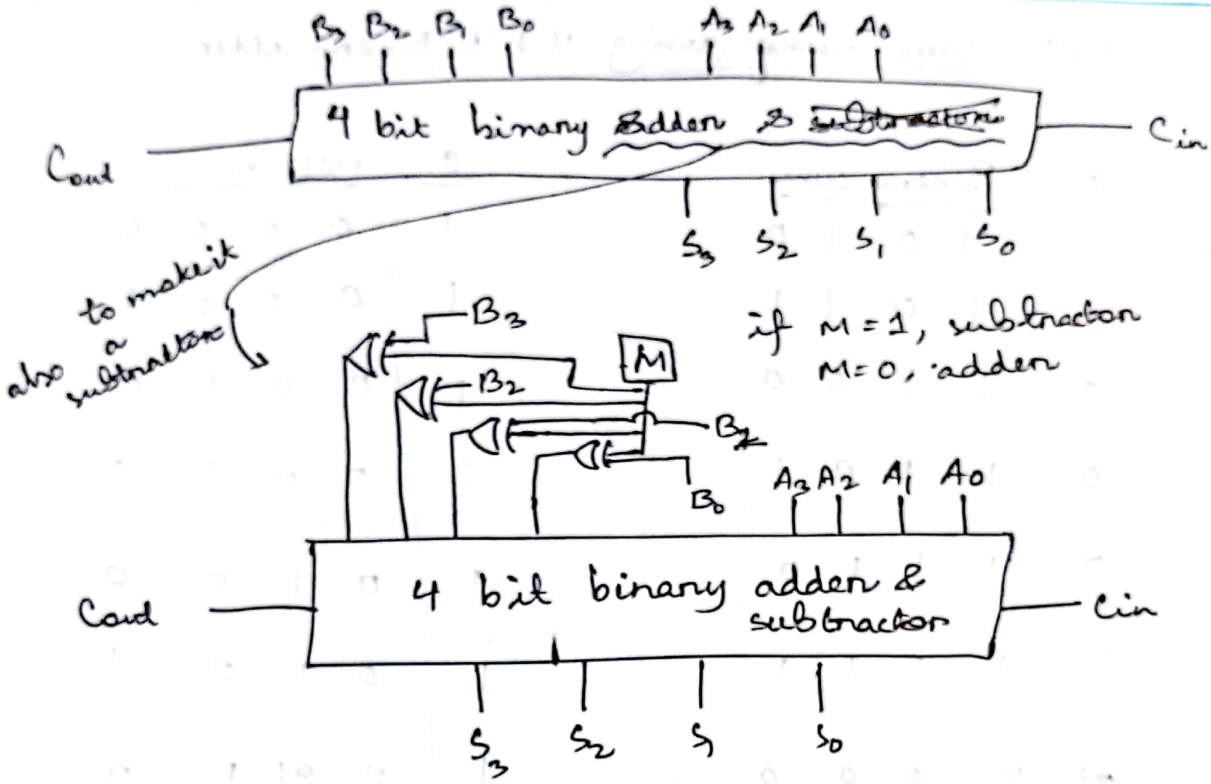
we need to fix this carry

$C = K + B_8 B_4 + B_8 B_2$



Topic Name : _____ Day : _____

Time : Date : / /



using a 4 bit adder as both adder & subtractor

Implementing MUX with 4 bit binary adder :

+ comparator +

in next class

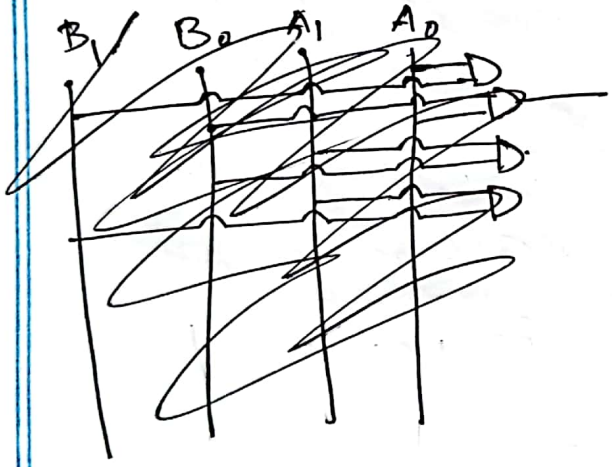
Topic Name : CSE 1203 (SH) Day : Sunday

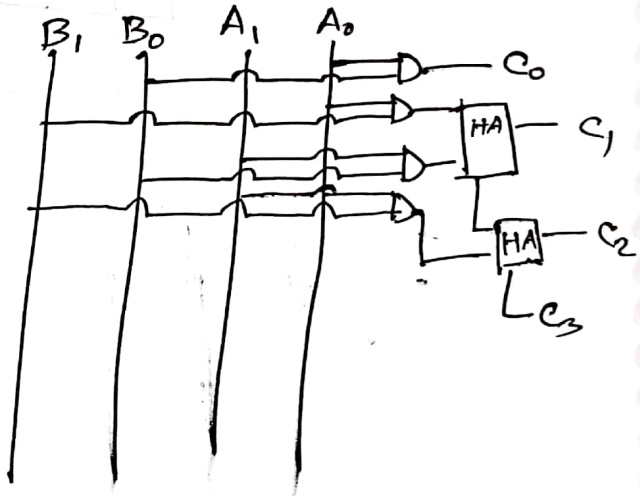
Time : 9.40 Date : 17/09/23

multiplication for boolean expressions :

$$\begin{array}{r} B_1 \quad B_0 \\ (\times) \quad A_1 \quad A_0 \\ \hline A_0B_1 \quad A_0B_0 \\ (+) \quad A_1B_1 \quad A_1B_0 \quad 0 \\ \hline \underbrace{A_1B_1}_{C_3} + \underbrace{A_0B_1}_{C_2} + \underbrace{A_1B_0}_{C_1} + \underbrace{A_0B_0}_{C_0} \end{array}$$

for n bit multiplication, product would be of $2n$ bits





Topic Name : _____ Day : _____

Time : Date : / /

B_2 B_1 B_0

A_1 A_0

A_0B_2 A_0B_1 A_0B_0

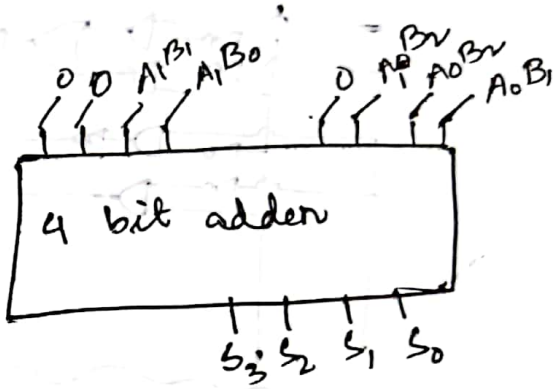
A_1B_2 A_1B_1 A_1B_0 0

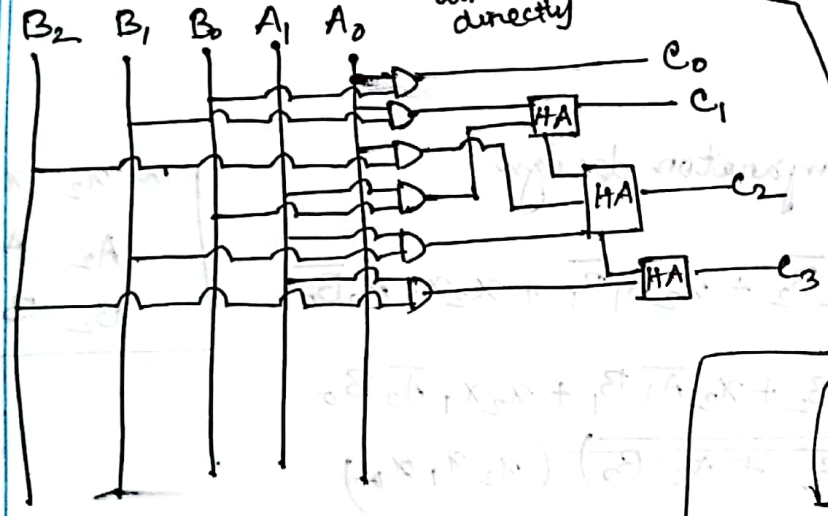
0 $A_1B_2 + (A_0B_2 + A_1B_1) + (A_0B_1 + A_1B_0) + A_0B_0$

0 0

→ it isn't needed as it's written directly

→ making it using 4 bit adder.





H.W

4 bit B_3 B_2 B_1 B_0

(A) 3 bit A_2 A_1 A_0

→ with H.A

→ with 4 bit parallel adder

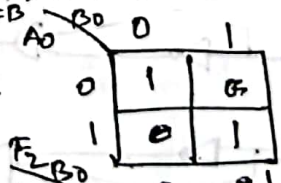
Comparator circuit designing

Flag bit : logical operational bit, returns '1' when logic = true

A_0	B_0	$A=B$	$A>B$	$A<B$
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

F_1 F_2 → flag bit F_3

$F_1 = A=B$



$F_1 = \overline{A_0 \oplus B_0}$

$= (A_0\overline{B_0} + \overline{A_0}B_0)$

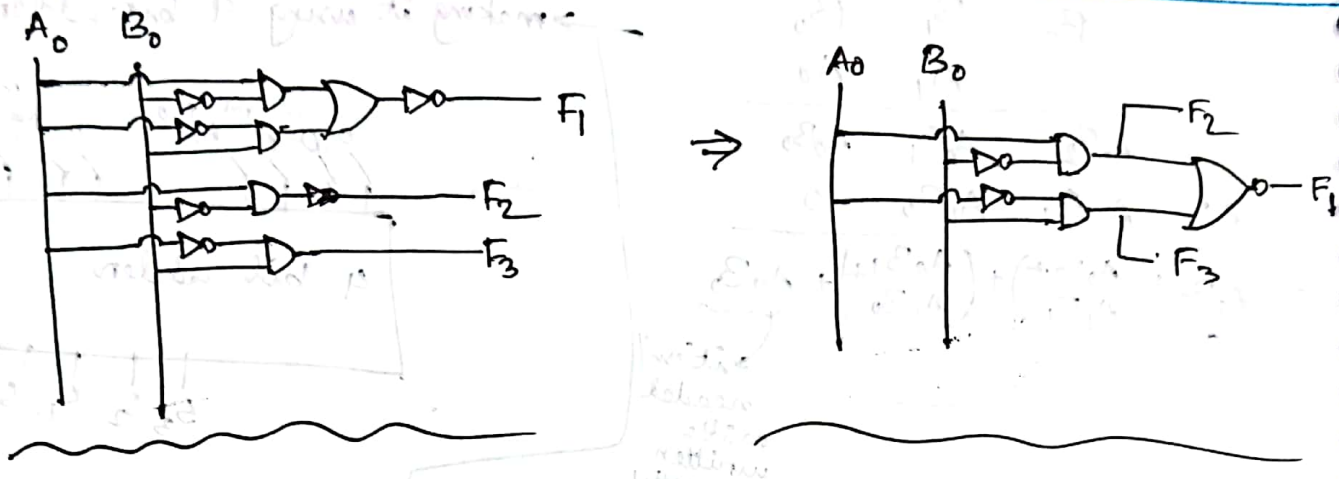
$F_2 = A_0\overline{B_0}$

$F_3 = \overline{A_0}\,B_0$

$A_3' B_3$ +

Topic Name : _____ Day : _____

Time : Date : / /



For 3 bits :

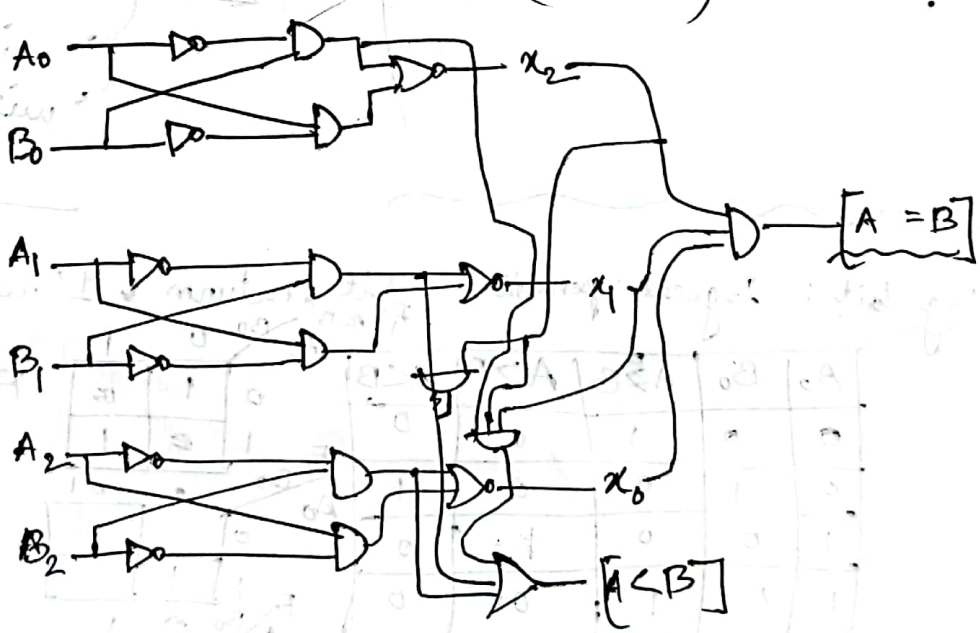
* 3 bit comparator design

*	x_2	x_1	x_0
	A_2	A_1	A_0
	B_2	B_1	B_0

$$F[A > B] = A_2\overline{B_2} + x_2 A_1 \overline{B_1} + x_2 x_1 A_0 \overline{B_0}$$

$$F[A < B] = \overline{A_2} B_2 + x_2 \overline{A_1} B_1 + x_2 x_1 \overline{A_0} B_0$$

$$F[A = B] = \overline{(A_0 B_0 + A_0 B_0)}\,(x_2 x_1 x_0)$$



Topic Name : CSE 1203 (SH) Day : Sunday

Time : 9.40 Date : 24/09/23

Continuing ALU

ALU: → comparator, → register] parts of ALU

ASSEMBLY LANGUAGE → (Assembler) → MACHINE LANGUAGE

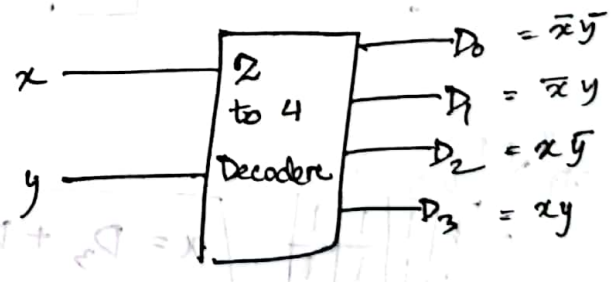
load →  → decoder → to machine's understanding

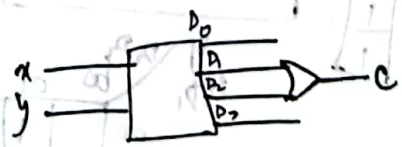
Decoder:

maximum could be $\leq 2^n$.

in 7 segment, we had to use atleast 3 input lines, to get maximum 8 outputs, but (even though we needed 7) we couldn't have used 2 input lines (maximum $4 = 2^2$)

C	b	x	y	D_0	D_1	D_2	D_3
0	0	0	0	1	0	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	0	1	0
1	0	1	1	0	0	0	1

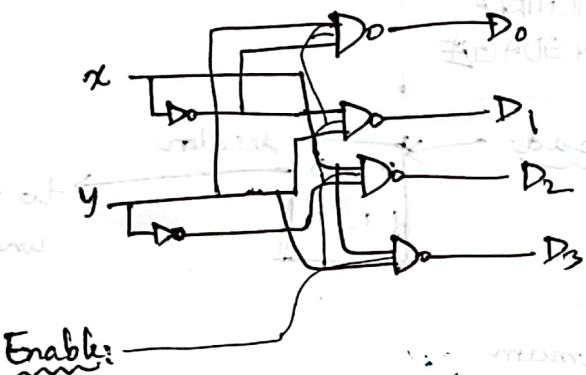


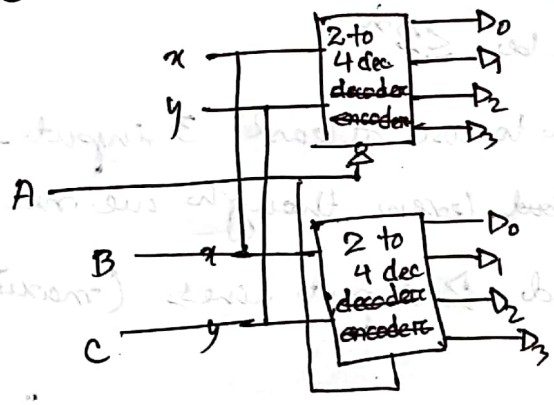


Topic Name : _____ Day : _____

Time : Date : / /

x	y	D_0	D_1	D_2	D_3
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0





Encoder

Decoder Encoder

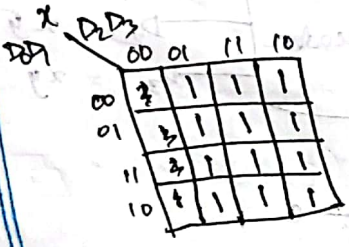
→ From n inputs, gives $\log_2 n$

D_0	D_1	D_2	D_3	x	y
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

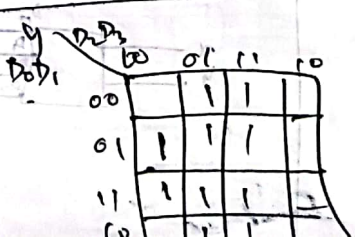
→ if we put 1 1 here, it shows 1 1 as output

D_0	D_1	D_2	D_3	x	y
1	0	0	0	0	0
x	1	0	0	0	1
x	x	1	0	1	0
x	x	x	1	1	1

→ x here means it'll be 1 in both 1/0 of it



$$x = D_3 + D_2$$



$$y = D_3 + \overline{D_2}\, D_1$$

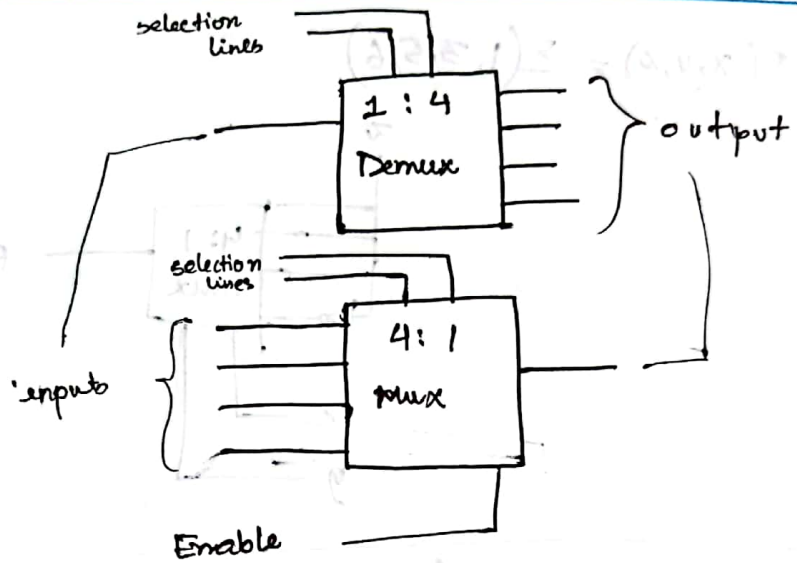
encoder

Topic Name : CSE 1203 (5H) Day : Sunday

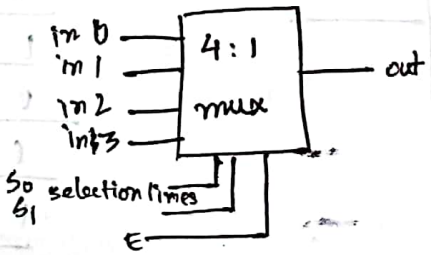
Time : 9.40 Date : 01/10/2023

Multiplexer :

Demultiplexer :



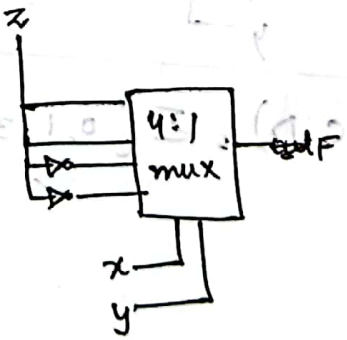
Designing a combinational circuit using MUX :



$F(x,y,z) = \Sigma(1,3,4,6)$

we assume any 2 ~~bit~~/variable as selection lines → do this table (x, y, z input)

then the 3rd variable will be input line

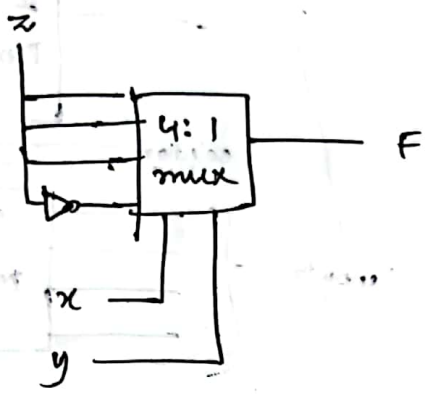


x	y	z	F	
0	0	0	0	
0	0	1	1	z
0	1	0	0	
0	1	1	1	z
1	0	0	1	$\bar{z}$
1	0	1	0	
1	1	0	1	$\bar{z}$
1	1	1	0	

Topic Name : _____ Day : _____

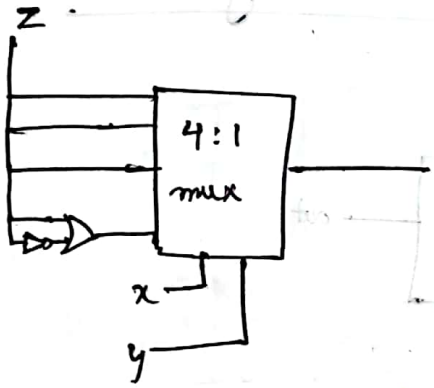
Time : Date : / /

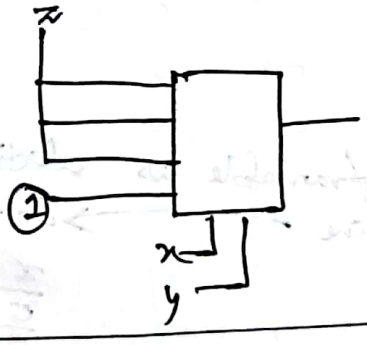
$F(x,y,z) = \Sigma(1,3,5,6)$



x	y	z	F	
0	0	0	0	
0	0	1	1	z
0	1	0	0	
0	1	1	1	z
1	0	0	0	
1	0	1	1	z
1	1	0	1	
1	1	1	0	$\bar{z}$

$F(x,y,z) = \Sigma(1,3,5,6,7)$



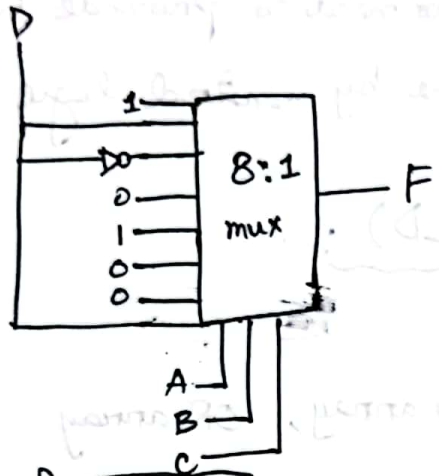


x	y	z	F	
0	0	0	0	
0	0	1	1	z
0	1	0	0	
0	1	1	1	z
1	0	0	0	
1	0	1	1	z
1	1	0	1	
1	1	1	1	$z+\bar{z}$

$F(A,B,C,D) = \Sigma(0,1,3,4,8,9,15)$

Topic Name : _____ Day : _____

Time : Date : / /



A	B	C	D	F	
0	0	0	0	1	
0	0	0	1	1	1
0	0	1	0	D	
0	0	1	1	D	D
0	1	0	0	1	
0	1	0	1	D	D
0	1	1	0	0	
0	1	1	1	0	0
1	0	0	0	1	1
1	0	0	1	1	
1	0	1	0	D	D
1	0	1	1	0	
1	1	0	0	0	0
1	1	0	1	0	
1	1	1	0	0	D
1	1	1	1	1	

Application of mux demux

- All communication lines

CT Syllabus :

- Parallel Adder to Mux : Demux

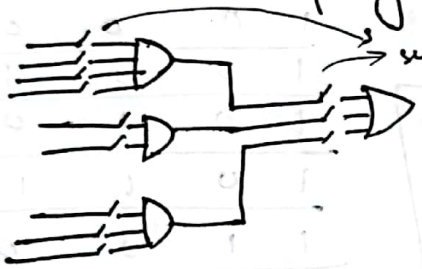
Topic Name : CSE 1203 (SH) Day : Tuesday

Time : 9.40 Date : 03 / 10 / 23

* when we design a system, we too need to provide some instructions *, these are done by control logic

Programmable logic Devices (PLD) :

↳ PLA : * can program AND array, OR array



switches → programmable : we can connect these switches

↳ PAL : * can program

↳ ROM : * can program only OR array

programmable : customizable

→ questions about PLA, PAL must contain these

eg: $F_0 = A + \bar{B}\bar{C}$

$F_1 = A\bar{C} + AB$

$F_2 = \bar{B}\bar{C} + AB$

$F_3 = \bar{B}C + A$

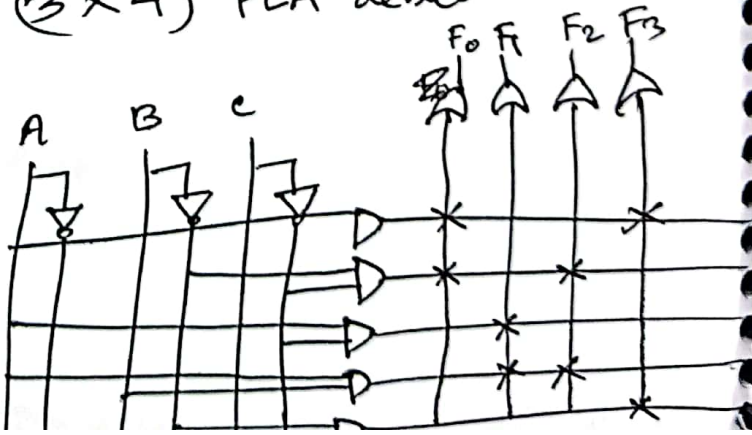
A, B, C → 3 inputs

$A, \bar{B}\bar{C}, A\bar{C}, AB, \bar{B}C$ → 5 product terms

F_0, F_1, F_2, F_3 → ~~3 × 4 PLA~~ 4 outputs

∴ (3 × 4) PLA device

personality matrix:



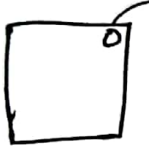
Topic Name : _____ Day : _____

Time : Date : / /

* we have to program with electrical signals

* PROM : Programmable ROM

* EPROM : Erasable PROM



→ if we light UV here, all the connections inside are erased

* EEPROM : Electrical EPROM

* deletes / erases not by UV but by specific electrical voltage

10.07.2023

CSE-1204 LAB

* Introduction with lab equipments and IC of different logic gates.

↳ IC:

identification:

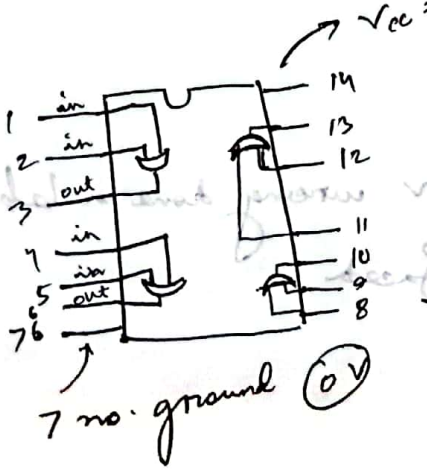
02 — NOR
00 — NAND
08 — AND gate
04 — NOT gate

SN 74 LS 04 — NOT gate
SN 74 HC 32 N — OR gate; N → Packaging: N-Plastic
SN → company code
74 → transistor logic – TTL (74/54)
→ emitter couple logic – ECL
→ MOS
→ CMOS

AND, OR, NOT } Basic
NAND, NOR } Universal
XNOR, XOR }

LS — low power Schottky
→ for voltage: 0–0.8 — 0; 2.5–5 — 1

HC — High Speed CMOS
→ 0–1.5 — 0
→ 3.5–5 — 1



Quad 2 input OR

CAUTION

* never take voltage level at exact 5.0 volt, take lower at 3.5 - 4.0 volt
* while wiring or placing IC or debugging, keep the connection with voltage opened.
* while debugging, VCC - grounding must be done first.

LAB REPORT

↳ Objective
↳ Introduction
↳ Required Equipment
↳ Truth table
↳ Circuit Diagram
↳ Discussion ☆ VVI → any mistake or wrong done in lab
→ any problem faced

DLD
CSE-1203
MMAH

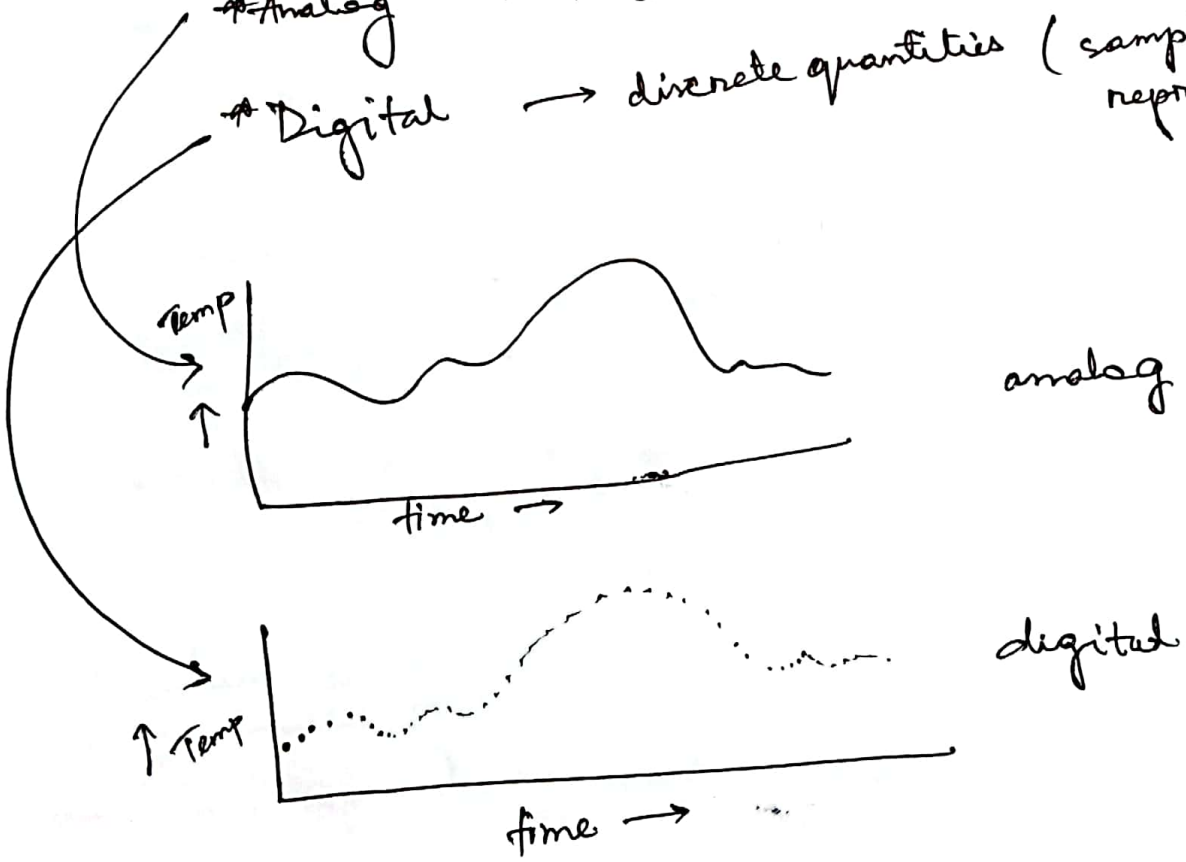
11.07.23

DLD

* Combinational logic circuits
* Sequential logic circuits

2 categories of ~~d~~ electronic circuits :

* ~~Analog~~ → continuous (most things in nature)
* Digital → discrete quantities (sample valued representation)



Advantage of digital :

* can be processed and transmitted more efficiently
*

Binary digits: BIT

- 0 or 1
- 1 → high voltage
- 0 → low voltage
- Group of bits are called codes
  - Being used to represent numbers, letters, symbols (i.e ASCII code), instructions and etc

Logic levels:

The voltages used to represent a 1 and 0 are called logic levels:

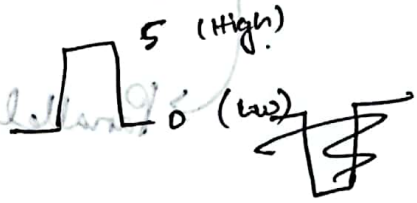
Digital waveforms:

Waveforms: series of pulses (pulse train)

- Periodic: period (T) = $T_1 = T_2 = T_3 = \ldots$

- non periodic:

ideal pulse

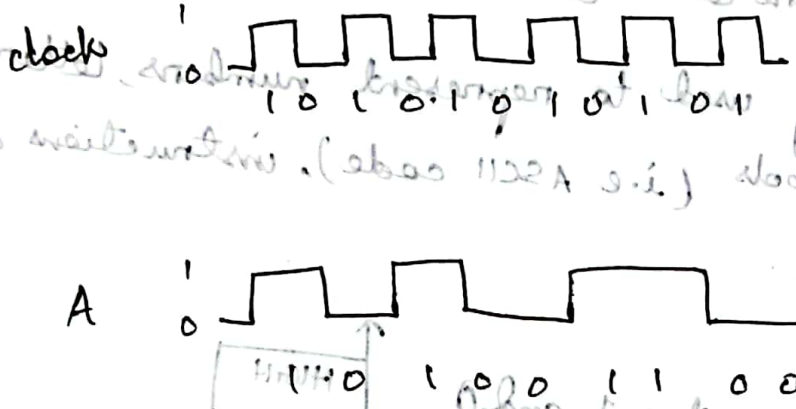


non ideal pulse:

Duty Cycle:

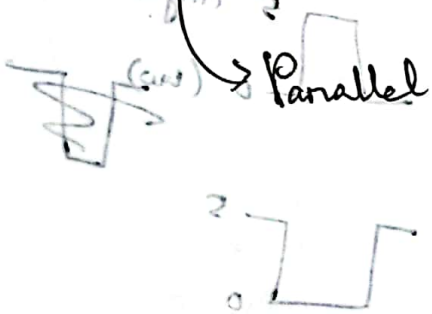
Ratio of the pulse width to period (T)

Waveform and Binary information:



Data Transfer:

- Serial — bits are sent one bit at a time
- Parallel — all the bits in a group are sent out on separate lines at the same time

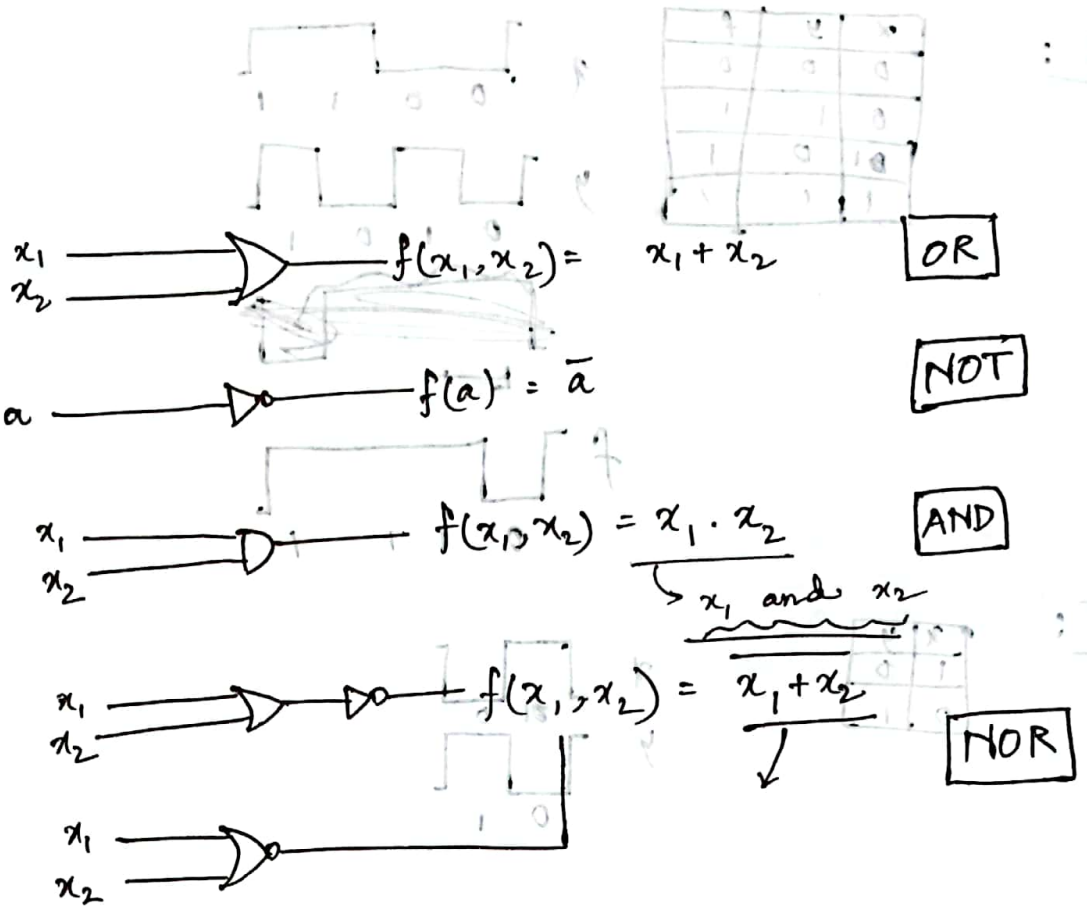


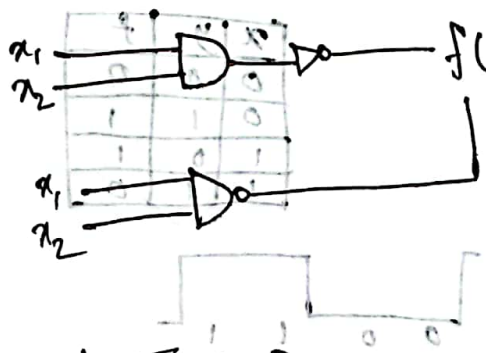
Serial over parallel:

- advantages
- disadvantages: takes more time

CSE-1205
DLD
MMAH

12.07.23



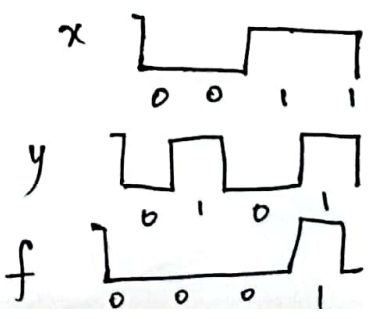


Logic Circuit Timing Diagram:

A graphical representation of input and output signal relationships over the time dimension of a truth table.

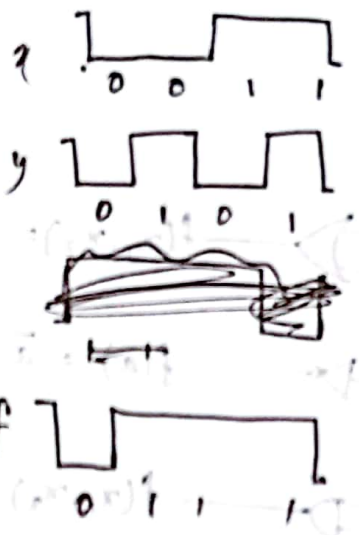
AND gate T.D:

x	y	f
0	0	0
0	1	0
1	0	0
1	1	1



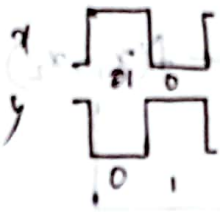
OR :

x	y	f
0	0	0
0	1	1
1	0	1
1	1	1

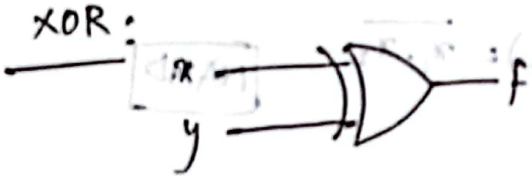


NOT :

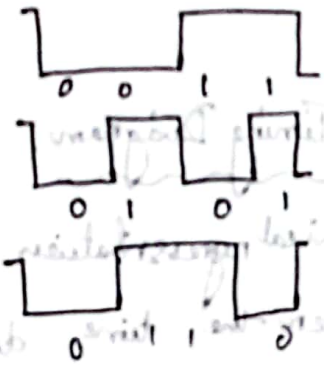
x	y
1	0
0	1



XOR :



x	y	f
0	0	0
0	1	1
1	0	1
1	1	0



* no matter how many inputs are given in XOR gate,
↳ an even number 1's or 0's would result in 0.

ASSIGNMENT | Digital Logic Design

We have a data on A in a register A

We want to transfer the data from A to B

x_1	x_2	x_3	Parity bit
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

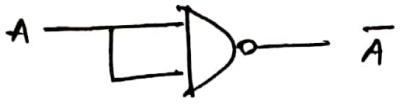
CSE 1204
LAB-DLD

17.07.23

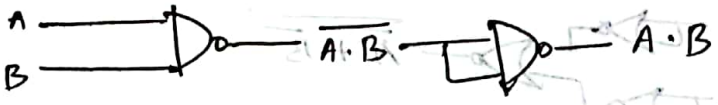
* Name of the experiment :

Bodean function implementation using universal gates.

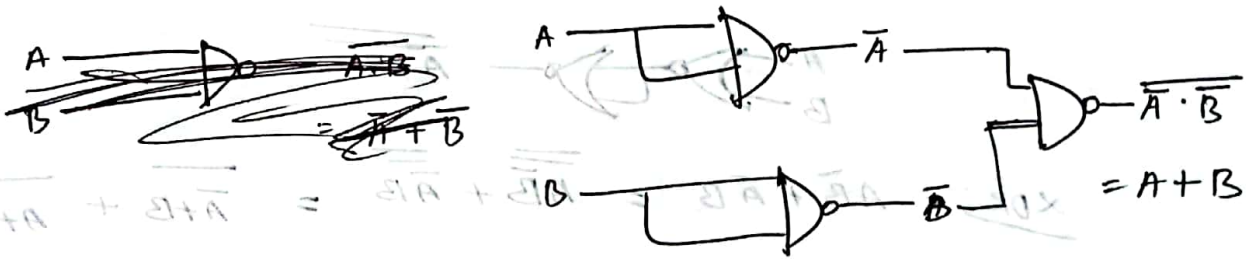
↳ NAND → NOT : $A \longrightarrow \overline{A}$



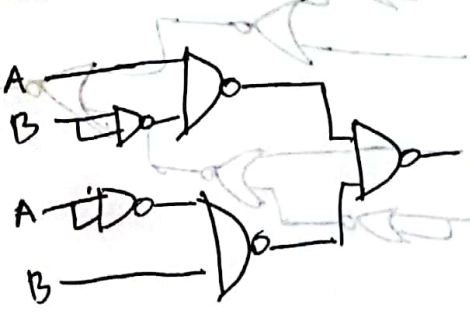
NAND → AND : $A, B \longrightarrow A \cdot B$



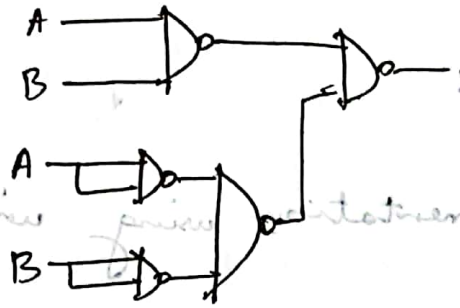
NAND → OR :



NAND → XOR : $A\overline{B} + \overline{A}B = \overline{\overline{A}\overline{B} + \overline{A}B} = \overline{A\overline{B}} \cdot \overline{\overline{A}B}$



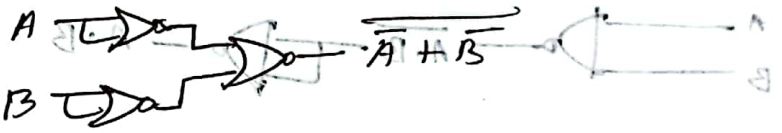
XNOR: $AB + \bar{A}\bar{B} = \overline{\overline{AB + \bar{A}\bar{B}}} = \overline{\overline{AB} \cdot \overline{\bar{A}\bar{B}}}$



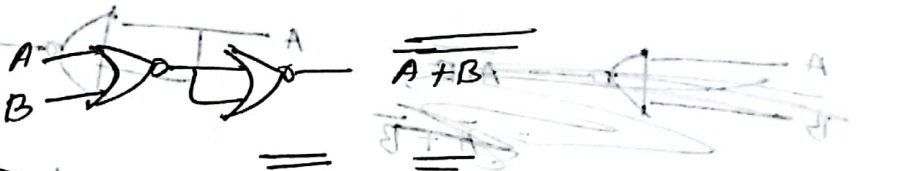
NOR

NOT:

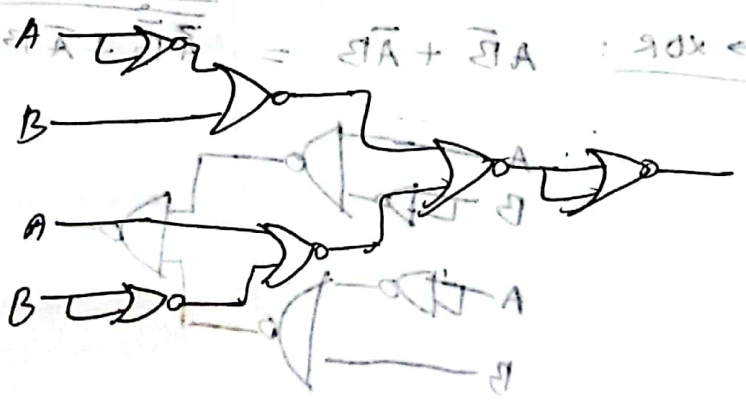
AND: $A \cdot B = \overline{\bar{A} + \bar{B}}$



OR: $A + B = \overline{\bar{A} \cdot \bar{B}}$

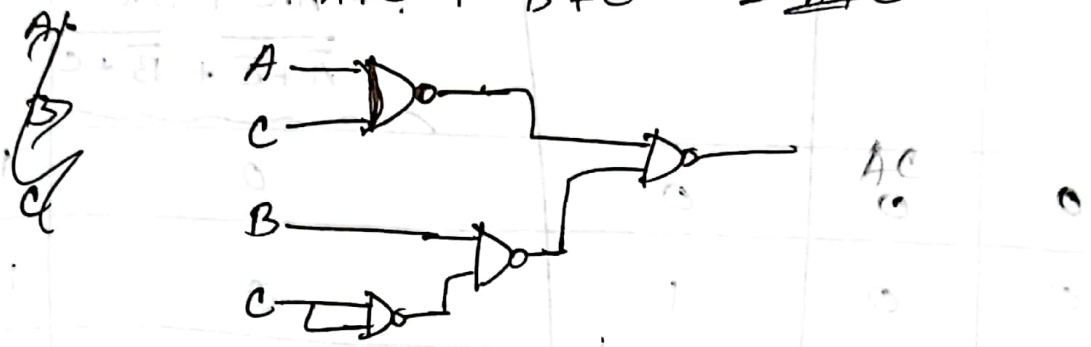


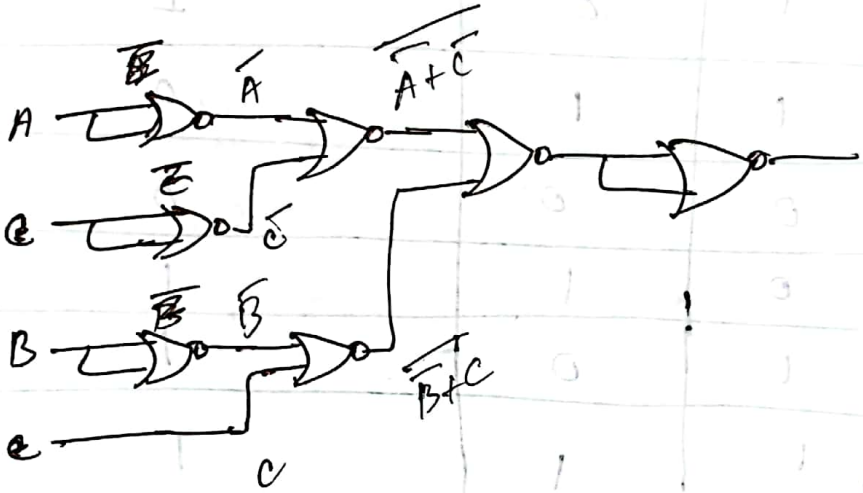
XOR: $A\bar{B} + \bar{A}B = \overline{\overline{A\bar{B}}} \cdot \overline{\overline{\bar{A}B}} = \overline{\bar{A} + B} + \overline{A + \bar{B}}$

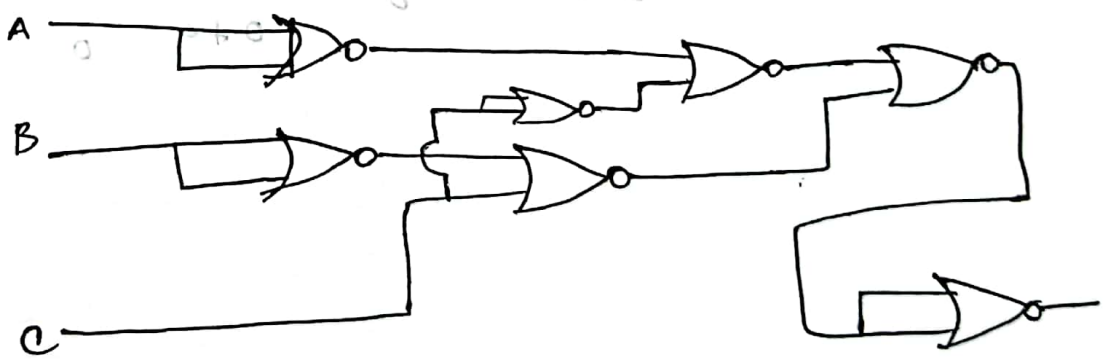


$F = A \cdot C + B \cdot \bar{C} = \overline{\overline{A \cdot C + B \cdot \bar{C}}} = \overline{\overline{A \cdot C} \cdot \overline{B \cdot \bar{C}}}$

$= \overline{\overline{A \cdot C}} + \overline{\overline{B \cdot \bar{C}}}$ $= \overline{\bar{A} + \bar{C}} + \overline{\bar{B} + C}$ $= \bar{A} + \bar{C}$







$F = A \cdot C + B \cdot \overline{C}$

$= \overline{\overline{A \cdot C} + \overline{B \cdot \overline{C}}}$

$= \overline{\overline{A} + \overline{C}} + \overline{\overline{B} + C}$

A	B	C	F	$\overline{A}$	$\overline{B}$	$\overline{C}$
0	0	0	0	1	1	1
0	0	1	0	1	1	0
0	1	0	1	1	0	1
0	1	1	0	1	0	0
1	0	0	0	0	1	1
1	0	1	1	0	1	0
1	1	0	1	0	0	1
1	1	1	1	0	0	0

0 + 0

0 + 0

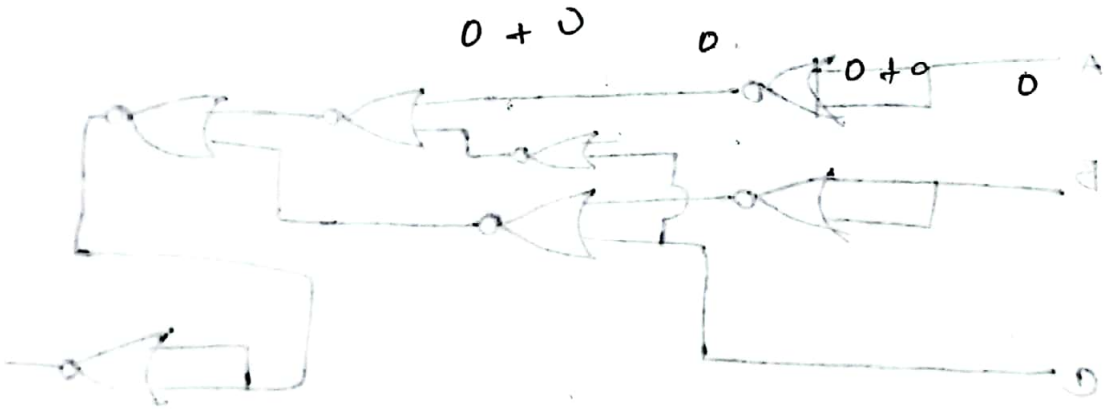
0

0 + 0

0

1 + 0

0 + 1



CSE 1204
DLD

24.07.23

ALU :
↳ Arithmetic logic unit

⇒ Experiment name: Design and implementation of full adder and full subtraction

↳ Half adder :

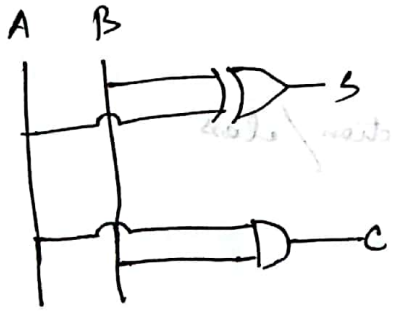
input: 2 → A, B

output: 2 → sum S, carry C

$S(A,B) = A \oplus B$

$C(A,B) = A.B$

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

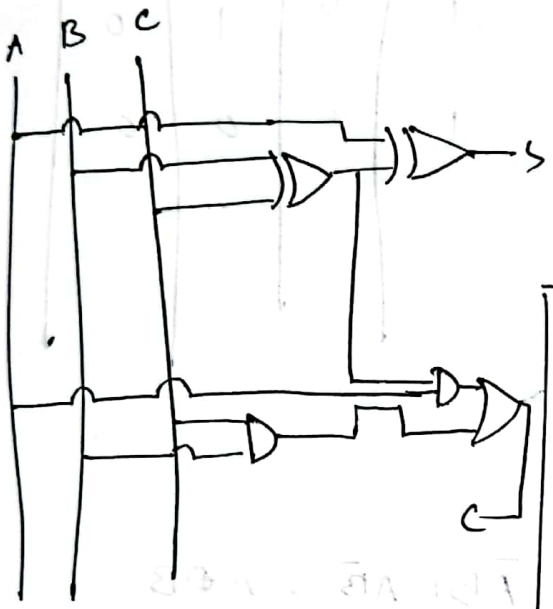


Full adder:

$S(A, B, C_{in}) = A \oplus B \oplus C_{in}$

$C_{out}(A, B, C_{in}) =$

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$

$= \bar{A}(\bar{B}C + B\bar{C}) + A(\bar{B}\bar{C} + BC)$

$= \bar{A} \cdot (B \oplus C) + A\overline{(B \oplus C)}$

$= A \oplus B \oplus C_{in}$

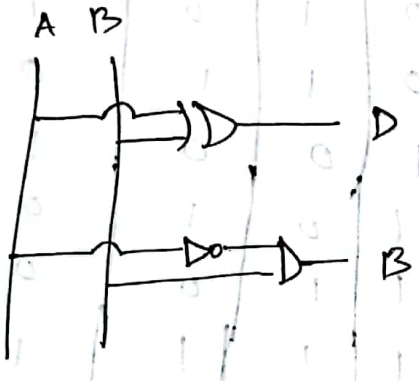
$C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$

$= BC(A + \bar{A}) + A(\bar{B}C + B\bar{C})$

$= BC_{in} + A(B \oplus C_{in})$

$=$

Half Subtractor :



A	B	D	B_{out}
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$D = \bar{A}B + A\bar{B} = A \oplus B$

$B_{out} = \bar{A}B$

Full subtractor :

$D = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$

$= \bar{A}(\bar{B}C + B\bar{C}) + A(\bar{B}\bar{C} + BC)$

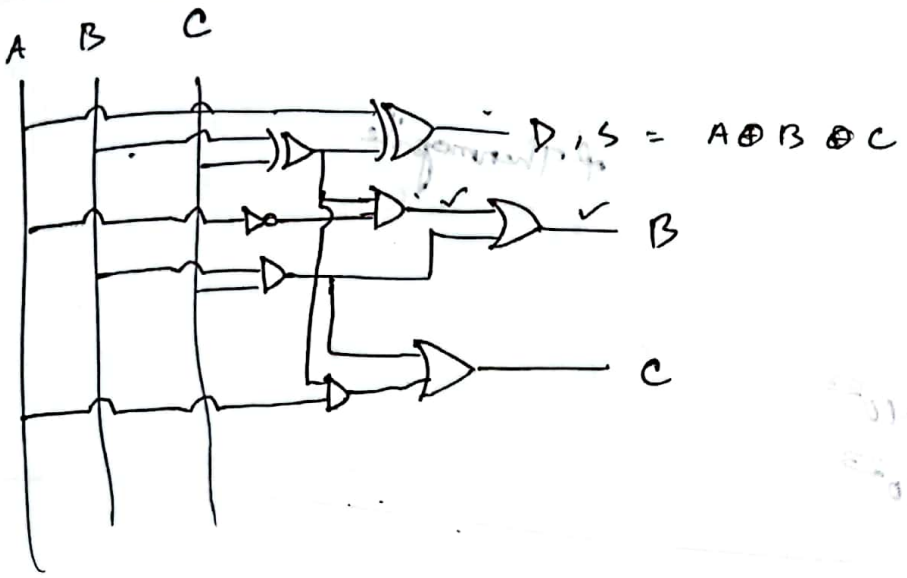
$= A \oplus B \oplus C$

$B = \bar{A}\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC + ABC$

$= BC(A + \bar{A}) + \bar{A}(\bar{B}C + B\bar{C})$

$= BC + \bar{A}(B \oplus C)$

A	B	C_{in}	D	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1


A
B
C
D, S = A ⊕ B ⊕ C
B
C

CSE 1203
MMAH

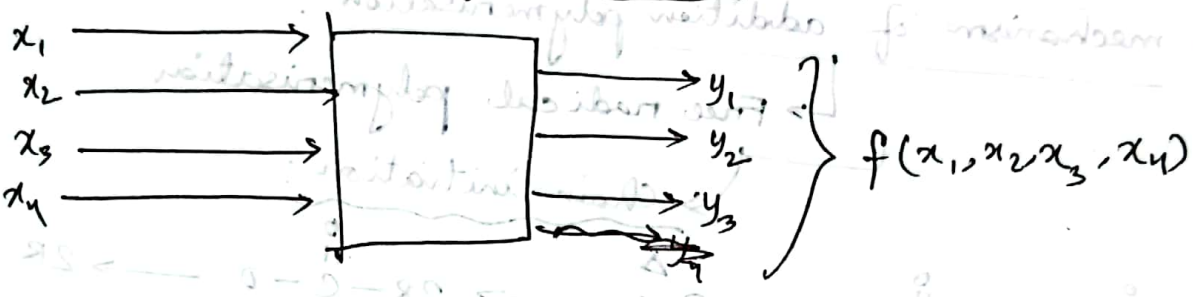
25.07.23

Logic circuit

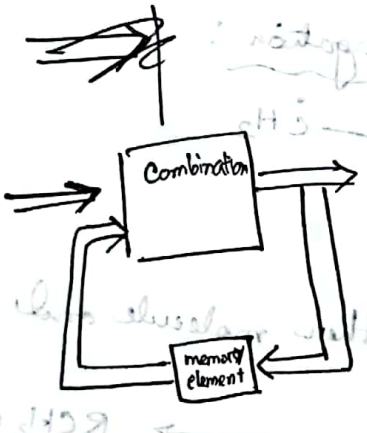
- combinational logic circuit
- sequential logic circuit

CLC:

instant input, instant output



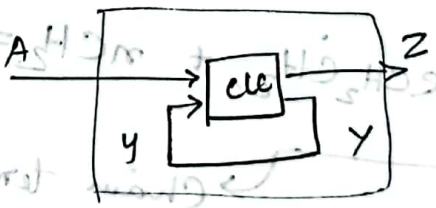
SLC:

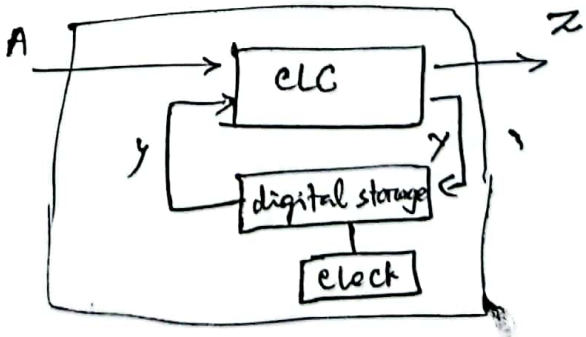


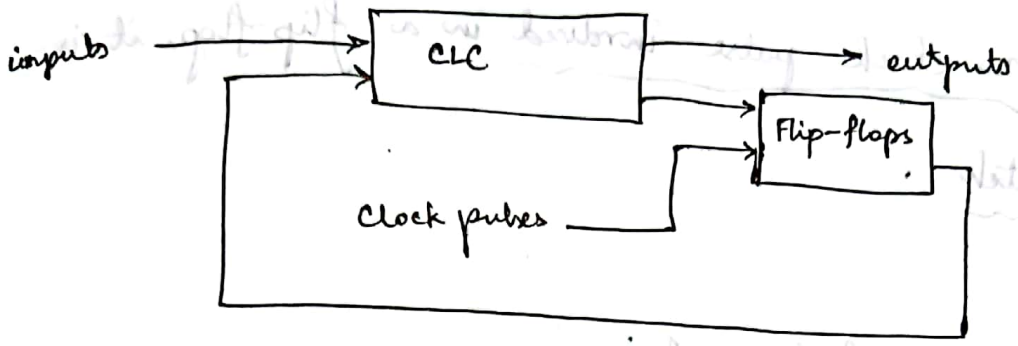
every digital system have CLC, most systems encountered in practice also include storage

Sequential LC

- asynchronous
- synchronous







Fig> Synchronous clocked sequential circuit

→ may use many flip-flops to store as many bits as neeessery

Fig: Timing diagram of clock pulses

SR Latch: → Asynchronous LC : no clock pulse

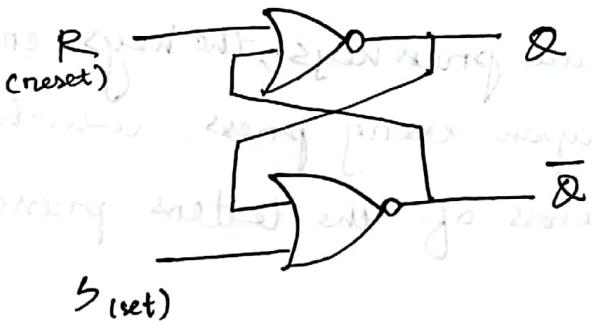


Fig : SR Latch

S	R	Q	$\bar{Q}$	
1	0	1	0	if $(\frac{S}{0}, \frac{R}{0})$ comes, it will be as it was before
0	0	1	0	
0	1	0	1	
0	0	0	1	same as before
1	1	0	0	

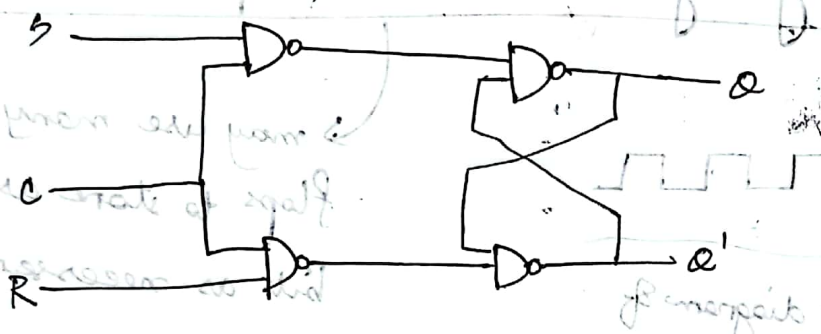
output Q and $\bar{Q}$ are always opposite to each other

Function Table,
~~Truth table~~

* Logic gates are defined by truth tables.
Flipflops are defined by behaviour tables / function tables.

if there is no clock pulse involved in a flip-flop, it is called a latch.

SR latch with control input:



Logic diagram

Uses of SR latch:

→ in keyboards, when we press keys, the keys create a shake / shock upon every press, which might result in repetitiveness of the letters pressed.

LAB 5
CSE 1204

Binary (B) | Gray (G)

	B_3	B_2	B_1	B_0	G_3	G_2	G_1	G_0
m_0	0	0	0	0	0	0	0	0
m_1	0	0	0	1	0	0	0	1
m_2	0	0	1	0	0	0	1	1
m_3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

Kanawgh

Bodean function :

K-Map:

B_3B_2 \ B_1B_0	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

$G_0 = \Sigma(1, 2, 5, 6, 9, 10, 13, 14)$

G_0:

B_2B_3 \ B_1B_0	01	10
	1	1
	1	1
	1	1
	1	1

$G_0 = \overline{B_1} B_0 + B_1 \overline{B_0} = B_1 \oplus B_0$

$G_1 = \Sigma(2, 3, 4, 5, 10, 11, 12, 13)$

B_3B_2 \ B_1B_0	00	01	11	10
00			1	1
01	1	1		
11	1	1		
10			1	1

$G_1 = B_2 \overline{B_1} + B_1 \overline{B_2}$

$\Rightarrow G_1 = B_2 \oplus B_1$

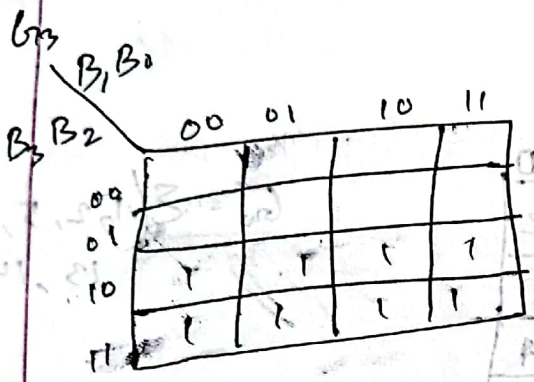
$G_2 = \Sigma(4, 5, 6, 7, 8, 9, 10, 11)$

B_3B_2 \ B_1B_0	00	01	11	10
00				
01	1	1	1	1
11				
10	1	1	1	1

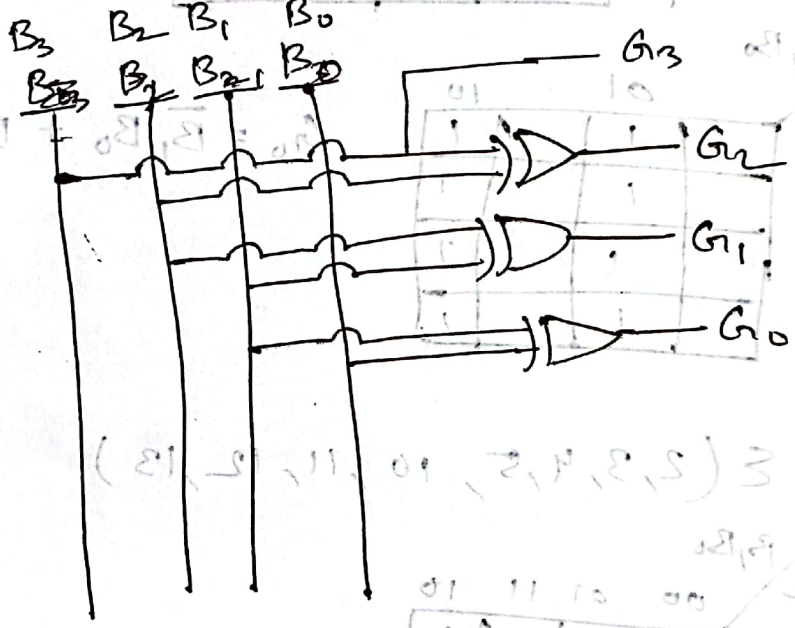
$G_2 = B_3 \overline{B_3} B_2 + B_3 \overline{B_2}$

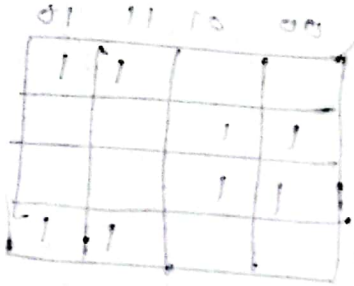
$= B_3 \oplus B_2$

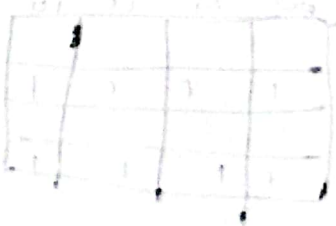
$G_3 = \Sigma(8, 9, 10, 11, 12, 13, 14, 15)$



$G_3 = B_3$







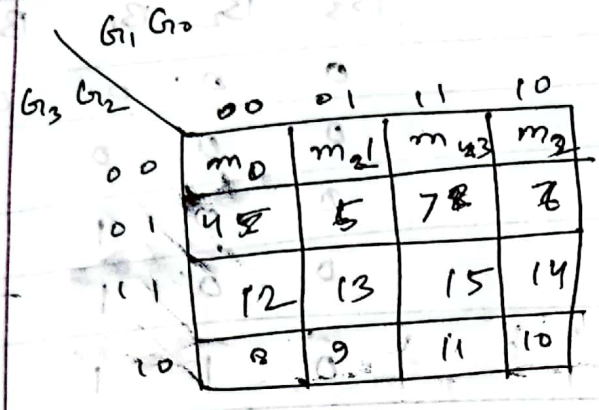
G_3	G_2	G_1	G_0	B_3	B_2	B_1	B_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
1	0	0	0	1	1	1	1
1	0	0	1	1	1	1	0
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	0	1	0	1	1
1	1	1	1	1	0	1	0

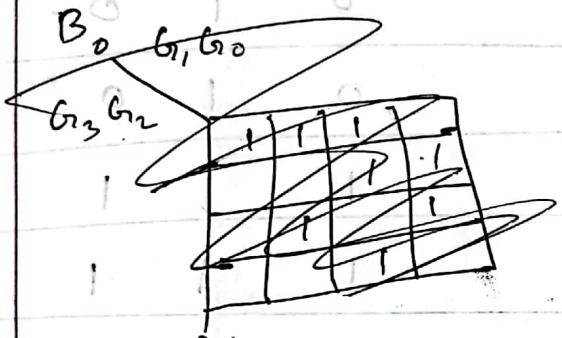
$B_0 = 1, 2, 4, 7, 8, 11, 13, 14$

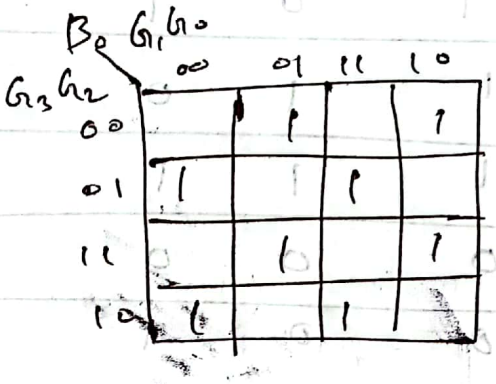
$B_1 = 2\ 3\ 4\ 5\ 8\ 9\ 14\ 15$

$B_2 = 4\ 5\ 6\ 7\ 8\ 9\ 10\ 11$

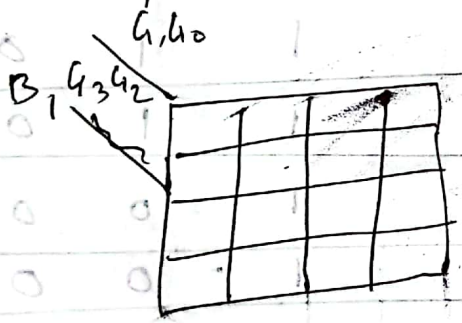
$B_3 = 8\ 9\ 10\ 11\ 12\ 13\ 14\ 15$

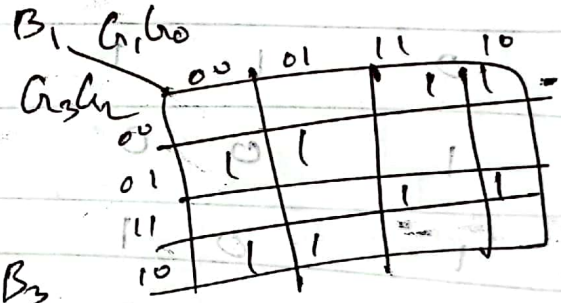


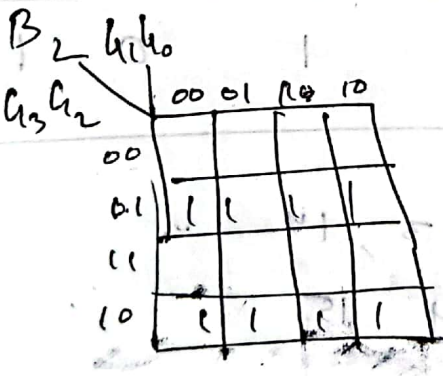




$B_0 = \overline{G_1}G_0 + G_1\overline{G_0}$ +







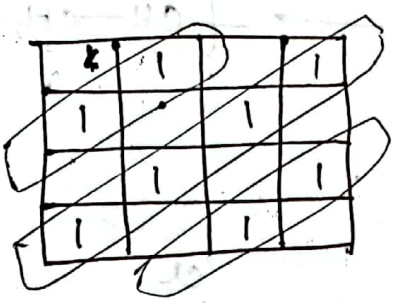
$\Rightarrow B_3 = G_3$

$\Rightarrow B_2 = \overline{G_3} G_2 + G_3 \overline{G_2} = G_3 \oplus G_2$

$\Rightarrow B_1 = \overline{G_3} \overline{G_2} G_1 + \overline{G_3} G_2 \overline{G_1} + G_3 G_2 G_1 + G_3 \overline{G_2} \overline{G_1}$

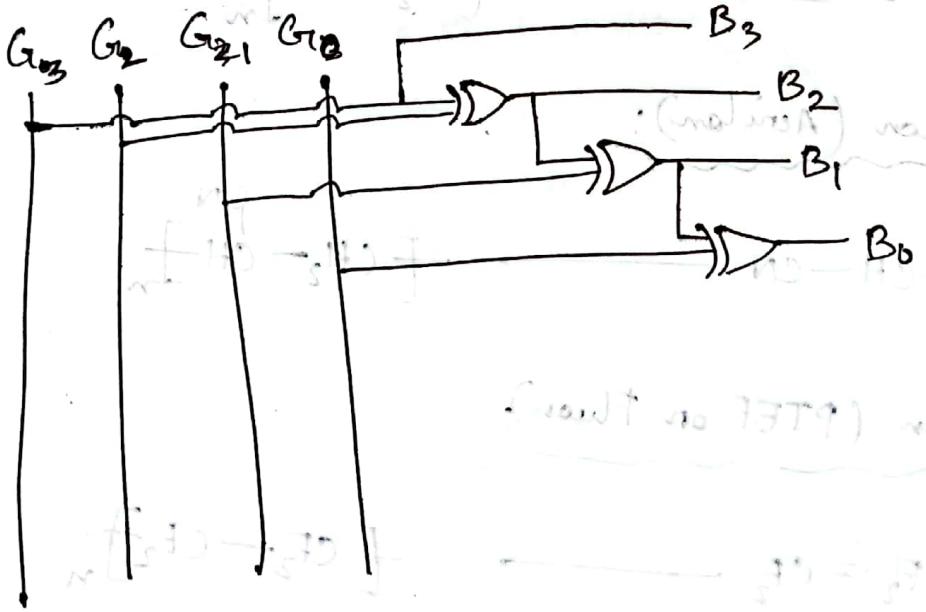
$= G_1 \oplus G_2 \oplus G_3$

$\Rightarrow B_0 =$



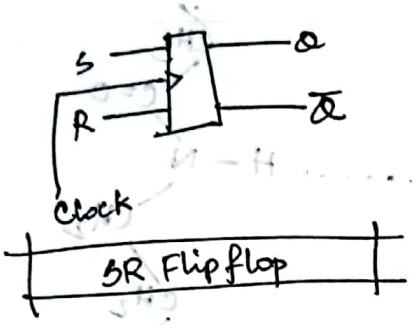
→ if such patterns occurs,

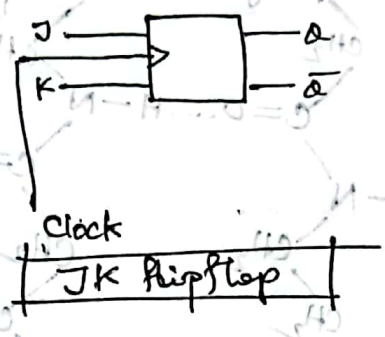
$B_0 = \overline{(G_0 \oplus G_1 \oplus G_2 \oplus G_3)}$

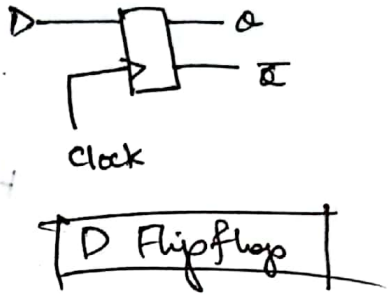


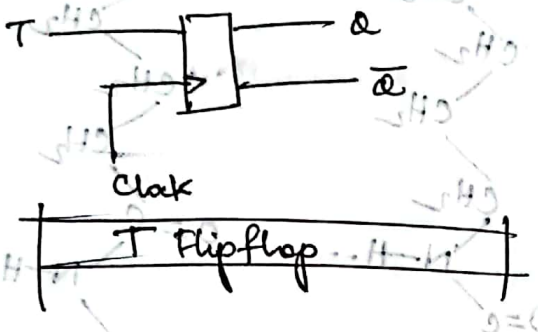
CS 203 MMPH

08.08.23







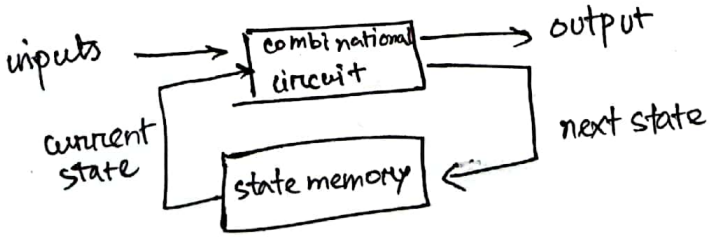


Synchronous sequential circuit analysis:

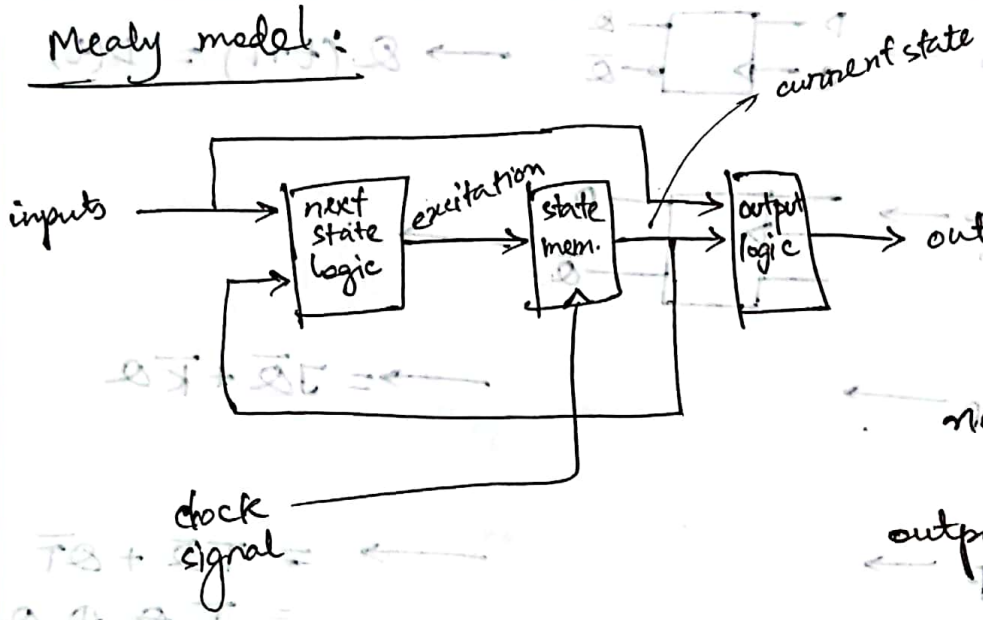
* The behaviour of a clocked seq. circuit depends on the inputs, outputs and the internal state.

* The outputs and the next state are both function on the applied inputs and the present state.

* The boolean expression that describe seque.. circuits



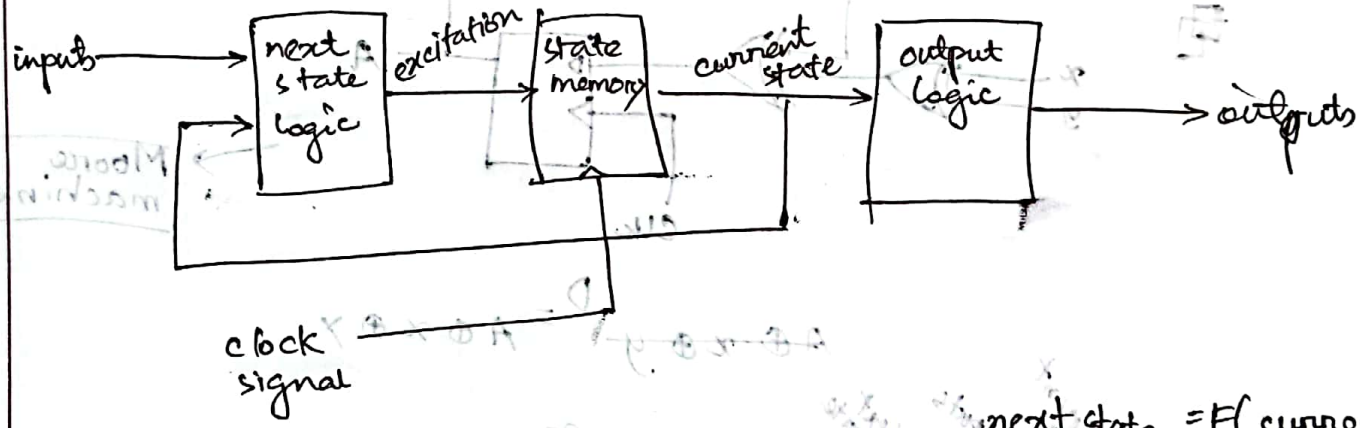
Mealy model :



next state = F(current state, input

outputs = G(current state, inputs)

Moore model :



next state = F(current state, inputs)

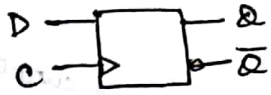
outputs = G(current state)

STATE DIAGRAM

↓

VVVVVI IMPORTANT

CSE 1203
MMATH

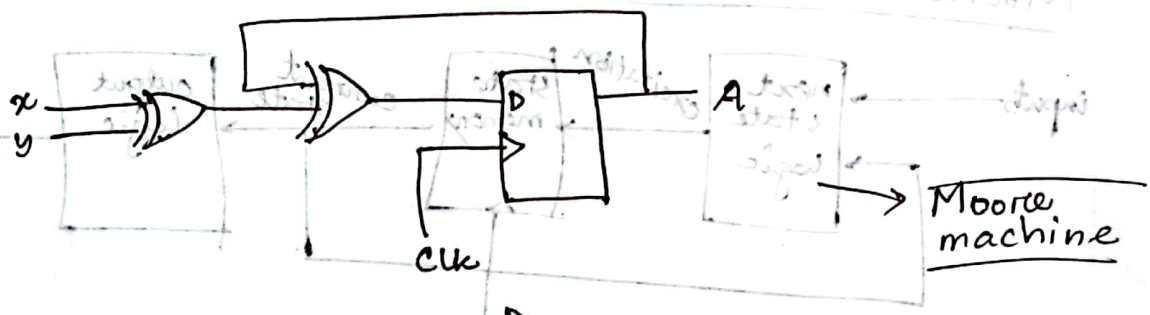
* D flipflop → 

→ $Q(t+1) = D(t)$

* SR Flipflop → 

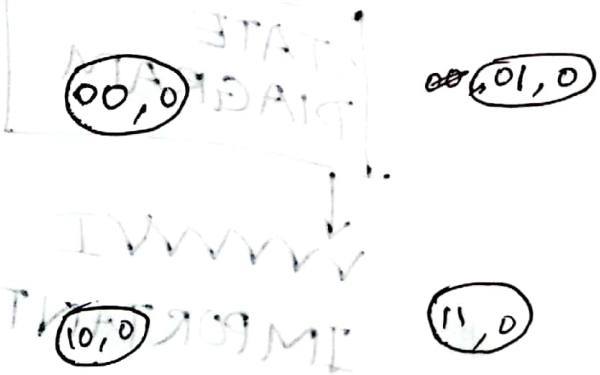
* JK Flipflop → $= J\bar{Q} + \bar{K}Q$

* T Flipflop → $= T\bar{Q} + Q\bar{T}$
$= T \oplus Q$

Ex 

$D = A \oplus x \oplus y$

Present state A	Inputs x	y	Next state A
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

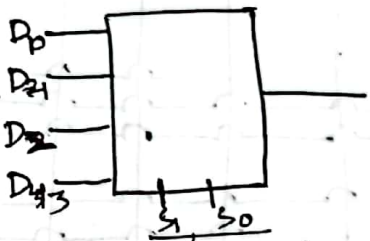


LAB 1
CSE 1204
DLD

14.08.25

Exp name: Design and implementation of multiplexer and demultiplexer circuits.

$S_0 \rightarrow S_1$



→ used to take multiple inputs and o give outputs one-by-one

→ selection bits

S_1	S_0	output
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

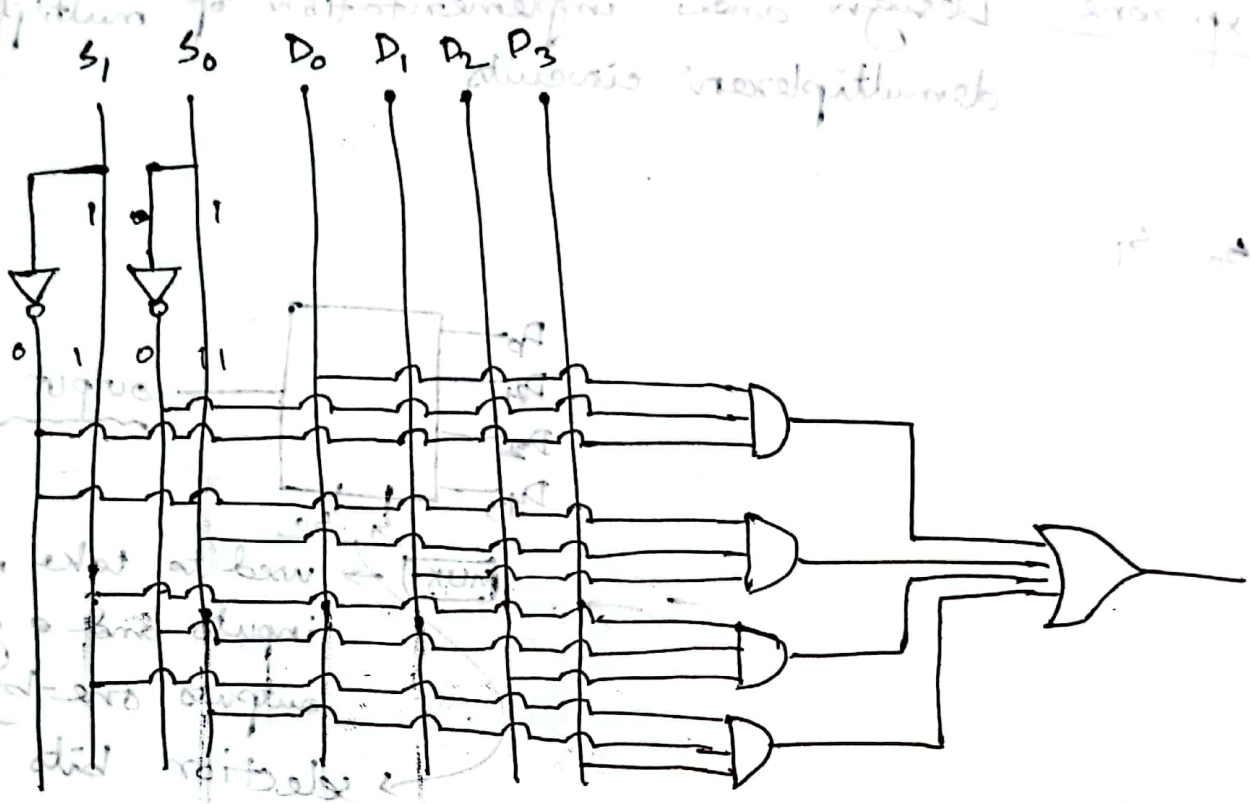
$Y_0 = (\overline{S_1}\,\overline{S_0} D_0 + \overline{S_1} S_0 D_1 + S_1 \overline{S_0} D_2 + S_1 S_0 D_3)$

$Y_0 = \overline{S_1}\,\overline{S_0} D_0$

$Y_1 = \overline{S_1} S_0 D_1$

$Y_2 = S_1 \overline{S_0} D_2$

$Y_3 = S_1 S_0 D_3$



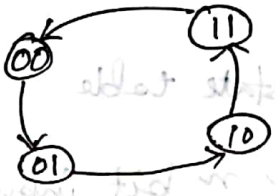
Demux :

S_1	S_0	I	D_0	D_1	D_2	D_3
0	0	1				
0	1	1				
1	0	1				
1	1	1				

Topic Name : CSE 1203 (MMAH) Day : Tuesday

Time : 9.40 Date : 29/08/23

* synchronous counter



state diagram

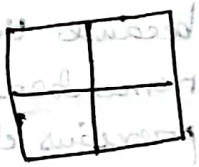
Q_1	Q_0	Q_1^+	Q_1^+
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0

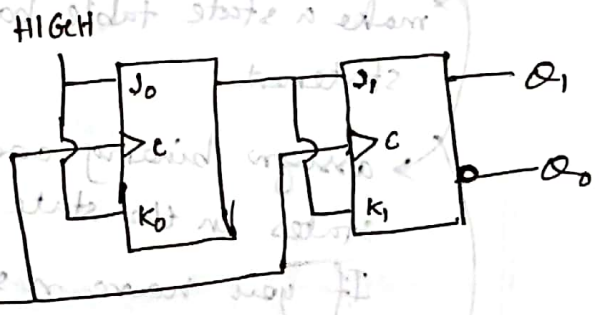
state table

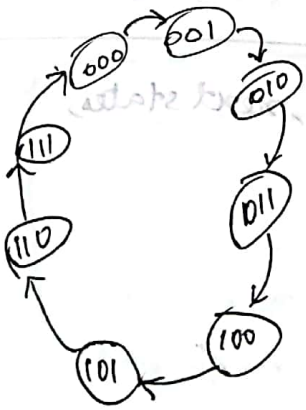
Present state		next state		FF input				Q_0
Q_1	Q_0	Q_1^+	Q_1^+	J_1	K_1	Q_0	J_0	K_0
0	0	0	1	0	X	1	1	X
0	1	1	0	1	X	0	X	1
1	0	1	1	X	0	1	X	X
1	1	0	0	X	1	1	X	X

001

Kmap :







Q_2	Q_1	Q_0	Q_2^+	Q_1^+	Q_0^+
0	0	0	0	0	1
0	0	1	0	1	0
0	1	1	1	0	0

$J_0 = 1$, $K_0 = 1$

$J_1 = Q_0$, $K_1 = Q_0$

$J_2 = Q_0 Q_1$, $K_2 = Q_0 Q_1$

Topic Name : CSE 1203 (MMAH) Day : Wednesday
Time : 9.40 Date : 30 / 08 / 23

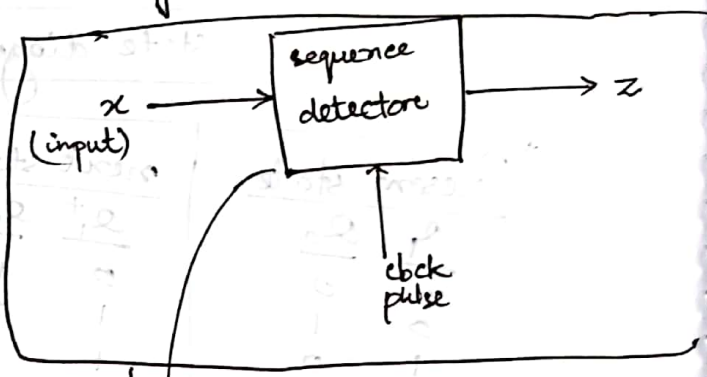
* Reverse Circuit designing (Sequential circuit design):

↳ Reverse engineering
↳ state diagram & state table

⇒ It takes n cycles to scan n bit inputs

sequence detector:

↳ overlapping
↳ non-overlapping



Steps

↳ make a state table based on probs statement.
↳ assign binary codes to the states in the state table.
If you have n states, your binary codes will have at least $\lceil \log_2 n \rceil$ digits and ffs
↳ for each ff, use the present state, next states,
↳ simplify the eqn.

sequence recognizer:

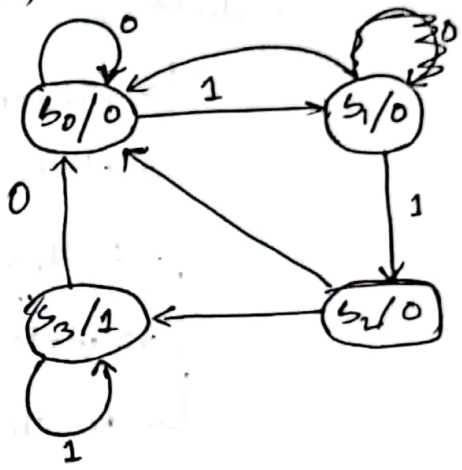
needs a sequential circuit because the circuit has to remember the inputs from previous clock cycles, in order to determine a match was found or not.

Topic Name : CSE 1203 (MMAH) Day : Tuesday

Time : Date : 12 / 09 / 23

* Design a circuit that detects a sequence of 3 consecutive 1's in a string of bits (with D ff)

→ State diagram :



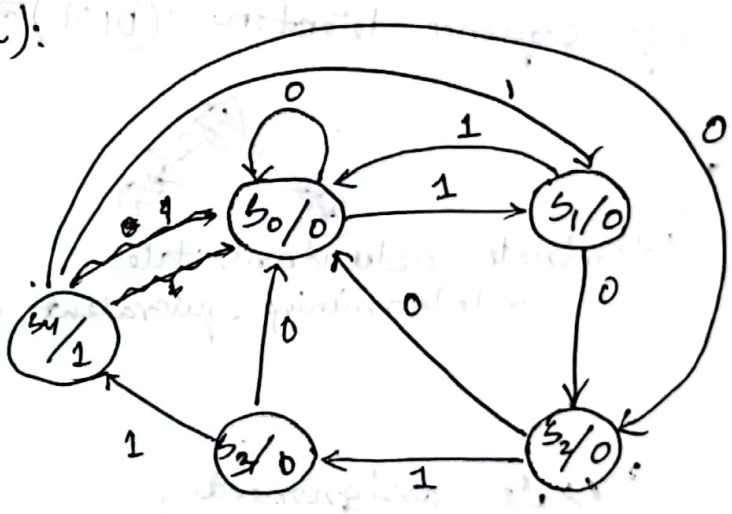
Moore

→ State table :

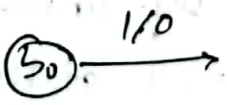
→ Excitation table :

→ Diagram (circuit) :

1011 detector :



Moore



Topic Name : CSE 1203 (MMAH) Day : Wednesday

Time : 9.40 Date : 13 / 09 / 23

minimizing state of sequential logic circuit

State reduction :

- understand specification
- state diagram
- tables (state table from state graph)
- state assigned minimization
- create state assigned table :

↳ avoid unnecessary states + remove unnecessary states

eg: sequence detector : (0101) ~~==~~ 11 (1101)

↳ eliminate redundant states

↳ determining equivalent states first :

↳ implication chart

State assignment :

given a seq. circuit with 3 states, 2 ffs, there are

$4 \times 3 \times 2 = 24$ states that can be assigned

Guidelines for state assignment

↳ ~~State~~

↳ states which have the same ~~level~~ next states for a given input should be given adjacent assignments

Topic Name : _____ Day : _____

Time : Date : / /

↳ states which are the next states of the same state should be given adjacent assignments.

↳ states which have the same output for a given input should be given adjacent assignments.

	x= 0	1	x= 0	1
a	a	c	0	0
b	d	f	0	1
c	e	a	0	0
d	d	b	0	1
e	b	f	1	0
f	c	e	1	0